

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CMC**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: CÔNG NGHỆ PHẦN MỀM
(INFO2011)**

**Tên đề tài:
WEB ĐỌC TRUYỆN CHỦ
ONLINE-CMC TRUYỆN**

Giảng viên: Phạm Tuấn Anh

Mã lớp học phần: N02

Nhóm sinh viên:

- | | |
|----------------------|-----------|
| 1. Nguyễn Thị Thùy | BAI252513 |
| 2. Trần Thị Kim Uyên | BAI250072 |
| 3. Nguyễn Hải Dương | BAI250020 |
| 4. Nguyễn Tuấn Thành | BAI252417 |
| 5. Vũ Viết Trí | BAI250063 |

Hà Nội, tháng 7 – 2026

PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	Mã sinh viên	Nội dung phụ trách
1	Nguyễn Thị Thùy	BAI252513	Điều phối hoạt động nhóm, quản lý backlog, phân chia nhiệm vụ, tổng hợp yêu cầu và review báo cáo; phụ trách thiết kế UI/UX phía người dùng, xây dựng trang chủ, trang chi tiết truyện, luồng đọc truyện và lịch sử đọc; phát triển chức năng phân trang và sắp xếp dữ liệu phía Backend; xây dựng kịch bản kiểm thử người dùng, thực hiện kiểm thử tích hợp và nghiệm thu tính năng.
2	Trần Thị Kim Uyên	BAI250072	Thiết kế PostgreSQL schema và migration; xây dựng các chức năng CRUD cho dữ liệu người dùng, truyện, chương, bình luận và các thực thể liên quan; thực hiện kiểm tra quyền truy cập, quyền sở hữu và tính hợp lệ của dữ liệu; phát triển Google OAuth/OTP, AI service, API upload Supabase và tính năng collaborator; tối ưu truy vấn, bảo đảm tính toàn vẹn dữ liệu, kiểm thử tầng database và soạn thảo tài liệu ERD.

3	Nguyễn Hải Dương	BAI250020	Phát triển các chức năng Frontend cho bình luận, theo dõi, đánh giá và thông báo; xử lý giao diện responsive trên nhiều kích thước màn hình; xây dựng chức năng Backend phục vụ hiển thị chi tiết truyện, chương, bình luận, đánh giá và lịch sử đọc; thực hiện validation dữ liệu, tích hợp API và đồng bộ dữ liệu giữa Frontend-Backend.
4	Nguyễn Tuấn Thành	BAI252417	Phụ trách thiết kế kiến trúc và phát triển hệ thống Backend, bao gồm authentication, JWT, RBAC, moderation, notification, audit log và Redis/BullMQ worker; xây dựng chức năng tìm kiếm và lọc dữ liệu truyện; phát triển logic cá nhân hóa và quản lý lịch sử đọc; tối ưu hiệu năng và tốc độ phản hồi API; cấu hình server, thiết lập CI/CD, logging, kiểm thử Backend và viết tài liệu kỹ thuật.
5	Vũ Viết Trí	BAI250063	Phụ trách thiết kế và phát triển giao diện dành cho Admin và Moderator, bao gồm dashboard, quản lý người dùng, truyện, báo cáo, bình luận, hồ sơ và audit log; xây dựng loading state, error state và route guard; thực hiện kiểm thử component, kiểm thử giao diện và kiểm thử phân quyền trên các màn hình quản trị.

MỤC LỤC

Danh sách bảng	iv
Danh sách hình vẽ	v
Danh mục từ viết tắt	vi
1 GIỚI THIỆU BÀI TOÁN	1
1.1 Bối cảnh và động cơ thực hiện	1
1.2 Phát biểu bài toán	1
1.2.1 Vấn đề của người đọc	1
1.2.2 Vấn đề của người đăng nội dung và đội ngũ vận hành	1
1.3 Mục tiêu của hệ thống	2
1.4 Đối tượng sử dụng và bên liên quan	3
1.5 Phạm vi chức năng	4
1.5.1 Chức năng trong phạm vi	4
1.5.2 Ngoài phạm vi hoặc chưa đủ minh chứng	5
1.6 Yêu cầu phi chức năng	6
1.7 Phân tích nhu cầu người dùng theo hướng sản phẩm	7
1.7.1 Tầm nhìn sản phẩm và phạm vi MVP	7
1.7.2 Personas	7
1.7.3 Scenarios và hành trình chính	9
1.7.4 User stories, backlog và acceptance criteria	9
2 PHÂN TÍCH LUỒNG HOẠT ĐỘNG	12
2.1 Tổng quan tác nhân và nguyên tắc phân quyền	12
2.2 Luồng của khách và người dùng	12
2.2.1 Khám phá và đọc truyện khi chưa đăng nhập	12
2.2.2 Đăng ký, đăng nhập và duy trì phiên	13
2.2.3 Theo dõi, lịch sử và tương tác	15
2.2.4 Mở khóa chương trả phí bằng Tinh thạch	15
2.3 Luồng của Uploader	16
2.3.1 Tạo truyện và nhập nội dung	16
2.3.2 Quản lý chương và cộng tác viên	16
2.4 Luồng của Moderator	17

2.5	Luồng của Admin	18
2.6	Luồng báo cáo vi phạm	19
2.7	Truy vết luồng đến endpoint và source	20
2.8	Quy tắc xử lý lỗi và an toàn luồng	21
2.9	Kết luận chương	21
3	TRIỂN KHAI HỆ THỐNG	22
3.1	Kiến trúc phần mềm tổng thể	22
3.1.1	Lựa chọn mô hình kiến trúc	22
3.1.2	Công nghệ và trách nhiệm	23
3.2	Triển khai frontend	23
3.2.1	Cấu trúc thư mục	23
3.2.2	Điều hướng và bảo vệ route	24
3.2.3	Lớp gọi API và quản lý phiên	25
3.2.4	Mở khóa chương ở frontend	26
3.2.5	Trạng thái giao diện và trải nghiệm	27
3.3	Triển khai backend	27
3.3.1	Cấu trúc phân lớp	27
3.3.2	Pipeline request	29
3.3.3	Xác thực, phân quyền và audit	30
3.3.4	Kiểm soát truy cập và mở khóa chương trả phí	31
3.3.5	Tác vụ nền và dịch vụ AI	31
3.4	Triển khai cơ sở dữ liệu	31
3.4.1	Mô hình dữ liệu	31
3.4.2	Connection pool, index và transaction	35
3.5	Cấu hình và triển khai	35
3.5.1	Biến môi trường	35
3.5.2	Topology triển khai	35
3.6	Kiểm thử và kết quả xác minh	37
3.6.1	Chiến lược kiểm thử	37
3.6.2	Kết quả tổng hợp	37
3.6.3	Nhóm 1: Xác thực người dùng	39
3.6.4	Nhóm 2: Upload truyện và kiểm duyệt	41
3.6.5	Nhóm 3: Chức năng người dùng	43
3.6.6	Kiểm thử thủ công bổ sung	46
3.7	Kết luận chương	47
4	CỘNG TÁC TRÊN GITHUB VÀ SỬ DỤNG CÔNG CỤ AI	48

4.1	Kho mã nguồn và quy ước cộng tác	48
4.2	Issues, commits, pull requests, milestones và releases . . .	48
4.3	Code review, kiểm thử tự động và DevOps	50
4.4	Nhật ký sử dụng công cụ và tác tử AI	51
5	LIÊN HỆ LÝ THUYẾT ENGINEERING SOFTWARE PRODUCTS	53
5.1	Phương pháp đối chiếu	53
5.2	Phân tích sâu các quyết định chính	55
5.2.1	Giới hạn MVP và quản lý phạm vi	55
5.2.2	Kiến trúc đủ dùng thay vì microservices sớm	55
5.2.3	AI-assisted development và trách nhiệm kỹ thuật	55
6	KẾT QUẢ THỰC HIỆN	57
6.1	Sản phẩm và kịch bản MVP	57
6.2	Đánh giá chức năng, cơ sở dữ liệu và kiến trúc	58
6.3	Hạn chế được xác nhận	58
6.4	Hướng phát triển	58
	Tài liệu tham khảo	59

DANH SÁCH BẢNG

1.1	Mục tiêu của hệ thống	2
1.2	Đối tượng sử dụng và nhu cầu chính	3
1.3	Yêu cầu phi chức năng và giải pháp đáp ứng	6
1.4	Persona đại diện của CMC Truyện	8
1.5	Các scenario nghiệp vụ đại diện	9
1.6	Backlog đại diện và tiêu chí nghiệm thu	10
2.1	Ma trận quyền theo vai trò được thể hiện trong source . .	12
2.2	Luồng quản trị chính và dấu vết cần lưu	18
2.3	Truy vết luồng nghiệp vụ đến API và module hiện thực . .	20
3.1	Stack công nghệ được xác nhận từ package và cấu hình dự án	23
3.2	Trách nhiệm các thư mục frontend chính	24
3.3	Phân lớp backend và nguyên tắc phụ thuộc	28
3.4	Nhóm biến môi trường và nguyên tắc bảo mật	36
3.5	Liên kết bàn giao và minh chứng triển khai	36
3.6	Công cụ và phạm vi kiểm thử theo tầng	37
3.7	Kết quả kiểm thử và build đã thực chạy	38
3.8	Test case nhóm xác thực người dùng	39
3.9	Test case nhóm upload truyện và kiểm duyệt	41
3.10	Test case nhóm chức năng người dùng	43
3.11	Kịch bản kiểm thử thử công bổ sung	46
4.1	Đối chiếu các cơ chế cộng tác trên GitHub	49
4.2	AI gợi ý, phân nhóm kiểm chứng/sửa/loại bỏ và quyết định của nhóm	51
5.1	Đối chiếu quyết định của nhóm với lý thuyết Sommerville .	54
6.1	Đối chiếu kết quả MVP với mục tiêu sản phẩm	57

DANH SÁCH HÌNH VẼ

1.1	Sơ đồ ngữ cảnh và các tác nhân chính	3
2.1	Luồng khám phá và đọc truyện của Guest	13
2.2	Trình tự xác thực và sử dụng JWT	14
2.3	Vòng đời kiểm duyệt của truyện	16
2.4	Luồng Moderator xử lý truyện chờ duyệt	17
3.1	Kiến trúc logic của hệ thống theo source hiện tại	22
3.2	Cấu trúc thư mục frontend	24
3.3	Đoạn code chính về route và route guard ở frontend	25
3.4	Đoạn code chính về Axios interceptor ở frontend	26
3.5	Cấu trúc thư mục backend	27
3.6	Pipeline xử lý request ở backend	29
3.7	Đoạn code chính cấu hình middleware và mount router	29
3.8	Đoạn code chính về xác thực JWT và RBAC	30
3.9	Lược đồ ERD của cơ sở dữ liệu CMC Truyện	32
3.10	Cấu trúc script, migration và schema cơ sở dữ liệu	33
3.11	Đoạn SQL chính về bảng truyện, chương và ràng buộc	34
4.1	Kho mã nguồn GitHub của dự án CMC Truyện	48
4.2	GitHub Actions với các lần chạy test và deploy thành công	50

DANH MỤC TỪ VIẾT TẮT

AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CSDL	Cơ sở dữ liệu
CSS	Cascading Style Sheets
DOM	Document Object Model
FE	Frontend – Phần giao diện phía người dùng
BE	Backend – Phần xử lý phía máy chủ
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
RBAC	Role-Based Access Control – Kiểm soát truy cập theo vai trò
REST	Representational State Transfer
SPA	Single-Page Application – Ứng dụng web một trang
SQL	Structured Query Language
UI/UX	User Interface/User Experience – Giao diện/Trải nghiệm người dùng
URL	Uniform Resource Locator

Chương 1. GIỚI THIỆU BÀI TOÁN

1.1. Bối cảnh và động cơ thực hiện

Nhu cầu đọc truyện trực tuyến ngày càng gắn với khả năng tìm kiếm nhanh, tiếp tục đúng vị trí đã đọc và sử dụng ổn định trên nhiều thiết bị. Khi số lượng truyện, chương, bình luận và báo cáo tăng, cách quản lý thủ công dễ phát sinh chậm trễ, sai sót và khó truy vết.

Từ thực tế đó, nhóm xây dựng ứng dụng web **CMC Truyện**. Hệ thống hướng đến hai mục tiêu chính: tạo trải nghiệm đọc thuận tiện cho người dùng và cung cấp quy trình đăng tải, kiểm duyệt, quản trị nội dung rõ ràng cho đội ngũ vận hành.

1.2. Phát biểu bài toán

1.2.1. Vấn đề của người đọc

Các vấn đề chính của người đọc gồm:

- Khó tìm kiếm và chọn truyện khi dữ liệu tăng;
- Không đồng bộ lịch sử, vị trí đọc và truyện đang theo dõi;
- Thiếu tùy chỉnh hiển thị và thông báo chương mới;
- Bình luận rác hoặc nội dung vi phạm làm giảm chất lượng cộng đồng.

1.2.2. Vấn đề của người đăng nội dung và đội ngũ vận hành

Người đăng nội dung cần quản lý truyện, chương và theo dõi trạng thái kiểm duyệt. Người kiểm duyệt cần xử lý hàng chờ, báo cáo vi phạm và phản hồi cho người đăng. Quản trị viên cần quản lý tài khoản, vai trò, nội dung và nhật ký thao tác. Vì vậy, hệ thống không chỉ thực hiện các chức năng thêm, sửa, xóa mà còn phải bảo đảm xác thực, phân quyền, quy trình trạng thái và khả năng truy vết.

1.3. Mục tiêu của hệ thống

Bảng 1.1: Mục tiêu của hệ thống

Mã	Mục tiêu	Kết quả mong đợi
M1	Cung cấp trải nghiệm khám phá và đọc truyện thuận tiện	Có trang tìm kiếm, chi tiết truyện, đọc chương, bảng xếp hạng và URL định danh theo slug.
M2	Cá nhân hóa trải nghiệm người dùng	Có lịch sử đọc, chương đã đọc, theo dõi truyện, thông báo và tùy chọn đọc.
M3	Bảo đảm quy trình xuất bản có kiểm duyệt	Truyện mới của Uploader ở trạng thái chờ; Moderator/Admin có thể duyệt, yêu cầu sửa hoặc từ chối.
M4	Bảo vệ thao tác theo đúng thẩm quyền	JWT xác thực phiên; middleware RBAC bảo vệ route; frontend có route guard tương ứng.
M5	Hỗ trợ quản trị và truy vết	Có quản lý người dùng, truyện, báo cáo, từ khóa nhạy cảm và audit log.
M6	Có khả năng kiểm chứng chất lượng kỹ thuật	Có kiểm thử backend, frontend và quy trình build production; kết quả thực chạy được trình bày ở Chương 3.

Báo cáo mô tả hệ thống tại thời điểm hoàn thiện sản phẩm. Kết quả được ghi nhận dựa trên mã nguồn, kiểm thử, bản build và minh chứng vận hành tương ứng.

1.4. Đối tượng sử dụng và bên liên quan



Hình 1.1: Sơ đồ ngữ cảnh và các tác nhân chính

Hình 1.1 khái quát năm nhóm tác nhân tương tác với hệ thống. Trong đó, Guest là người chưa đăng nhập, không phải một vai trò được lưu trong bảng users.

Bảng 1.2: Đối tượng sử dụng và nhu cầu chính

Đối tượng	Nhu cầu	Giá trị hệ thống cung cấp
Guest	Khám phá nội dung trước khi đăng ký	Xem trang chủ, tìm kiếm, bảng xếp hạng, thông tin truyện và nội dung công khai.
User	Đọc lâu dài và tương tác	Lưu lịch sử, theo dõi, đánh giá, bình luận, nhận thông báo và tùy chỉnh trải nghiệm đọc.
Uploader	Đưa nội dung lên hệ thống	Tạo và chỉnh sửa truyện, nhập nội dung, quản lý chương và cộng tác viên theo quy trình duyệt.
Moderator	Duy trì chất lượng nội dung	Duyệt truyện chờ và xử lý báo cáo, bình luận hoặc hồ sơ theo phạm vi được giao.
Admin	Quản trị toàn hệ thống	Quản lý người dùng, vai trò, truyện, báo cáo, từ khóa, nhật ký và số liệu tổng quan.
Nhóm phát triển	Bảo trì và mở rộng sản phẩm	Duy trì kiến trúc, API, cơ sở dữ liệu, kiểm thử, tài liệu và cấu hình triển khai.

1.5. Phạm vi chức năng

1.5.1. Chức năng trong phạm vi

Phạm vi chức năng được xác định từ yêu cầu dự án, route frontend và router backend [1], gồm:

- Xác thực và tài khoản: Đăng ký, đăng nhập, đăng xuất, Google OAuth, quên/đặt lại mật khẩu bằng OTP và cập nhật hồ sơ;
- Khám phá truyện: Danh sách, tìm kiếm, lọc theo tag hoặc danh mục, bảng xếp hạng và trang chi tiết;
- Trải nghiệm đọc: Đọc chương theo URL thân thiện, lưu lịch sử, lưu chương đã đọc và tùy chọn hiển thị;
- Mở khóa chương trả phí: Hiển thị số dư Tinh thạch, kiểm tra quyền đọc, mở khóa chương bằng số dư nội bộ và lưu lịch sử giao dịch;
- Tương tác: Theo dõi, đánh giá sao, bình luận, trả lời, bình chọn và báo cáo vi phạm;
- Đăng nội dung: Tạo truyện, import tệp văn bản/EPUB, quản lý chương, tag và cộng tác viên;
- Kiểm duyệt: Hàng chờ truyện, yêu cầu sửa, từ chối/duyet, xử lý bình luận, hồ sơ và báo cáo;
- Quản trị: Quản lý user, role, trạng thái tài khoản, truyện, từ khóa, báo cáo và audit log;
- Tính năng hỗ trợ: Tóm tắt chương và gợi ý truyện bằng AI, notification, worker và hàng đợi tác vụ nền.

1.5.2. Ngoài phạm vi hoặc chưa đủ minh chứng

Các nội dung ngoài phạm vi hoặc chưa đủ bằng chứng gồm:

- Thanh toán, gói thuê bao, quảng cáo, bản quyền nội dung và chia sẻ doanh thu;
- Nạp Tinh thạch hoặc thanh toán qua cổng thanh toán thật; hệ thống hiện chỉ cấp số dư demo và ghi nhận giao dịch mở khóa nội bộ.
- Ứng dụng di động native hoặc chế độ đọc ngoại tuyến;
- Kiểm thử tải, kiểm thử xâm nhập và chỉ số SLA trên môi trường production;
- Khả năng chịu lỗi của Redis/worker và tính đúng đắn của hàng đợi khi hệ thống khởi động lại;
- Liên kết website production, video demo và dashboard giám sát thực tế;
- Số liệu người dùng thật, dữ liệu vận hành thật hoặc kết quả kinh doanh.

1.6. Yêu cầu phi chức năng

Bảng 1.3: Yêu cầu phi chức năng và giải pháp đáp ứng

Thuộc tính	Yêu cầu	Giải pháp/giới hạn hiện tại
Bảo mật	Route riêng tư phải được xác thực và phân quyền	JWT, Helmet, CORS, rate limiter, RBAC và audit middleware; chưa có kết quả penetration test.
Hiệu năng	Tránh response lớn và kết nối DB lặp lại	PostgreSQL pool, pagination, index, API danh sách chương không trả content; bundle frontend còn cảnh báo kích thước.
Khả dụng	Giao diện phản hồi trên các trạng thái tải/lỗi	Skeleton, optimistic UI có rollback ở một số tương tác; chưa có số liệu uptime.
Dễ bảo trì	Tách trách nhiệm và có quy ước source	Frontend theo các nhóm pages, components, services, contexts; backend theo routes, controllers, models, services, middleware và workers.
Khả năng mở rộng	Tác vụ dài không chặn request chính	Redis/BullMQ và process worker riêng; cần kiểm thử tích hợp với hạ tầng thật.
Truy vết	Thao tác quản trị quan trọng có lịch sử	Audit log lưu actor, action, entity và details đã làm sạch dữ liệu nhạy cảm.
Tương thích	Hoạt động như SPA trên hosting tĩnh	Vercel rewrite về index.html; API URL cấu hình qua biến môi trường.

1.7. Phân tích nhu cầu người dùng theo hướng sản phẩm

1.7.1. Tầm nhìn sản phẩm và phạm vi MVP

Từ các vấn đề đã trình bày ở phần trước, nhóm xác định tầm nhìn sản phẩm bằng một câu: *CMC Truyện là nền tảng đọc truyện trực tuyến sạch, nhanh, không có quảng cáo làm gián đoạn màn hình đọc, cho phép tùy chỉnh trải nghiệm và tự động lưu tiến độ để người dùng tiếp tục đọc thuận tiện.* Tầm nhìn này được cụ thể hóa trong docs/product/PRODUCT_DIRECTION.md và docs/product/PRODUCT_ANALYSIS.md.

MVP ban đầu được giới hạn ở ba năng lực cốt lõi: trình đọc có chuyển chương rõ ràng; tùy chỉnh sáng, sepia, tối và cỡ chữ; tự động lưu, khôi phục chương đang đọc. Tiêu chí kiểm chứng là người dùng có thể vào trang, tìm truyện, đọc và chuyển chương trong dưới 30 giây; khi quay lại, hệ thống mở đúng chương dở. AI tóm tắt, gợi ý cá nhân hóa, bình luận và quản trị nâng cao là lớp mở rộng, không phải điều kiện để giá trị đọc cốt lõi tồn tại.

1.7.2. Personas

Hai persona đại diện được tổng hợp từ docs/product/REQUIREMENTS.md và docs/product/PRODUCT_ANALYSIS.md. Đây là giả thuyết sản phẩm để định hướng thiết kế; nhóm chưa coi chúng là kết luận nghiên cứu người dùng nếu chưa có phỏng vấn hoặc số liệu sử dụng thực tế.

Bảng 1.4: Persona đại diện của CMC Truyện

Thuộc tính	Persona độc giả	Persona Up-loader/Moderator/Admin
Bối cảnh	Sinh viên hoặc người đi làm, đọc bằng điện thoại/laptop trong thời gian rảnh; thường theo dõi nhiều truyện.	Người đăng nội dung hoặc vận hành cộng đồng; cần xử lý truyện, chương, người dùng và báo cáo.
Mục tiêu	Tìm truyện nhanh; đọc lâu không mỏi mắt; lưu đúng tiến độ; nhận chương mới và tương tác.	Đăng và cập nhật nội dung nhanh; theo dõi trạng thái duyệt; giảm thao tác kiểm duyệt lặp lại; truy vết quyết định.
Khó khăn	Quảng cáo che nội dung, giao diện chói, mất vị trí đọc khi đổi thiết bị.	Khối lượng nội dung lớn, công cụ cũ chậm, khó rà soát bình luận/báo cáo thủ công.
Tiêu chí thành công	Tìm–đọc–chuyển chương nhanh, cấu hình đọc được lưu, quay lại đúng chương dở.	Quy trình tạo–duyet–công khai có trạng thái rõ, phân quyền đúng và có audit log.

1.7.3. Scenarios và hành trình chính

Bảng 1.5: Các scenario nghiệp vụ đại diện

STT	Tác nhân và mục tiêu	Luồng chính	Kết quả
1	Guest/User tìm và đọc truyện	Mở trang chủ, nhập từ khóa hoặc lọc; xem chi tiết; chọn chương; hệ thống tải nội dung và điều hướng trước/sau.	Đọc đúng truyện và chương mong muốn.
2	User theo dõi và đọc tiếp	Đăng nhập; theo dõi truyện; đọc chương; backend ghi lịch sử/tiến độ; lần truy cập sau khôi phục vị trí.	Truyện nằm trong tủ sách và không mất tiến độ.
3	Uploader tạo truyện	Mở dashboard; nhập metadata, ảnh bìa và tag; tạo truyện ở trạng thái pending; bổ sung chương theo quyền được cấp.	Truyện vào hàng chờ kiểm duyệt.
4	Moderator / Admin duyệt truyện	Mở hàng chờ; xem metadata và nội dung; chọn duyệt, yêu cầu sửa hoặc từ chối; hệ thống cập nhật trạng thái và ghi audit.	Quyết định có thể truy vết; nội dung chỉ công khai khi đạt yêu cầu.

1.7.4. User stories, backlog và acceptance criteria

Đặc tả dự án có 15 user story và product backlog PB01–PB19. Bảng 1.6 giữ lại các mục đại diện cho MVP và các luồng xuyên vai trò; trạng thái được đối chiếu với docs/product/REQUIREMENTS.md và mã nguồn hiện tại.

Bảng 1.6: Backlog đại diện và tiêu chí nghiệm thu

Mã	User story / tính năng	Ưu tiên	Trạng thái	Acceptance criteria rút gọn
PB01-02	Xem chi tiết và đọc chương	Cao	Hoàn thành	Trang chi tiết hiển thị meta-data/danh sách chương; route đọc trả đúng nội dung; điều hướng trước/sau không sai chương.
PB03-03B	Uploader tạo truyện, Moderator / Admin duyệt	Cao	Hoàn thành	Chỉ Uploader được tạo; truyện mới ở trạng thái chờ; người có quyền duyệt được ba nhánh quyết định; quyết định được ghi audit.
PB06	Tìm kiếm truyện	Cao	Hoàn thành	Tìm theo từ khóa và bộ lọc; kết quả đúng điều kiện; có trạng thái rõng/lỗi và giới hạn phân trang.
PB07-08	Đăng ký và đăng nhập	Cao	Hoàn thành	Dữ liệu được validation; mật khẩu băm; đăng nhập trả JWT; route bảo vệ từ chối token sai/hết hạn.
PB09-10	Theo dõi và lịch sử đọc	Trung bình	Hoàn thành	User có thể thêm/bỏ theo dõi; tiến độ được lưu theo tài khoản; lần sau tiếp tục đúng chương.
PB12	Tóm tắt chương bằng AI	Trung bình	Hoàn thành	Nút tóm tắt gọi API; ưu tiên cache RAM/DB; gọi Groq rồi Gemini khi cache miss; có timeout và fallback an toàn.
PB13	Gợi ý truyện cá nhân hóa	Trung bình	Hoàn thành	Chỉ user đăng nhập; đầu vào từ lịch sử đọc; ID do AI trả về được chuẩn hóa/kiểm tra; thiếu kết quả thì bù bằng truyện phổ biến.

Mã	User story / tính năng	Ưu tiên	Trạng thái	Acceptance criteria rút gọn
PB17- 19	Báo cáo, kiểm duyet nền và xử lý vi phạm	Cao/ Trung bình	Đang hoàn thiện	Báo cáo có trạng thái; worker dùng Redis/BullMQ xử lý nền; Admin/Moderator có thể xem, quyết định và truy vết.

Acceptance criteria là điều kiện hoàn thành task; kết quả đạt khi có bằng chứng test/build/audit log. Việc tách mô tả nhu cầu khỏi điều kiện kiểm chứng giúp tránh tình trạng hoàn thành code nhưng chưa trao giá trị thực tế.

Chương 2. PHÂN TÍCH LUỒNG HOẠT ĐỘNG

2.1. Tổng quan tác nhân và nguyên tắc phân quyền

Hệ thống sử dụng kiểm soát truy cập theo vai trò. Backend là lớp quyết định quyền cuối cùng thông qua `authenticateToken` và `authorizeRole`; route guard ở frontend chủ yếu cải thiện trải nghiệm bằng cách không hiển thị màn hình mà người dùng không được phép truy cập. Không được xem frontend guard là biện pháp bảo mật duy nhất, vì người dùng vẫn có thể tự gửi HTTP request đến API.

Bảng 2.1: Ma trận quyền theo vai trò được thể hiện trong source

Nhóm hành động	Guest	User	Uploader	Moderator	Admin
Xem truyện/chương công khai	Có	Có	Có	Có	Có
Đọc chương trả phí	Không	Có***	Có	Có***	Có
Tìm kiếm, bảng xếp hạng	Có	Có	Có	Có	Có
Lưu lịch sử, theo dõi, bình luận	Không	Có	Có	Có	Có
Tạo truyện/chương	Không	Không	Có	Không	Hạn chế*
Quản lý truyện được phép	Không	Không	Có	Không	Có
Duyệt truyện chờ	Không	Không	Không	Có	Có
Xử lý báo cáo/bình luận	Không	Không	Không	Có	Có
Quản lý vai trò/tài khoản/từ khóa	Không	Không	Không	Không	Có
Xem audit log	Không	Không	Không	Có**	Có

* Route tạo truyện hiện yêu cầu đúng role Uploader, trong khi một số route sửa/xóa cho phép Uploader hoặc Admin. Đây là khác biệt cần giải thích khi demo.

** Moderator chỉ được xem phạm vi audit phù hợp theo logic backend/test, không mặc nhiên có toàn quyền như Admin.

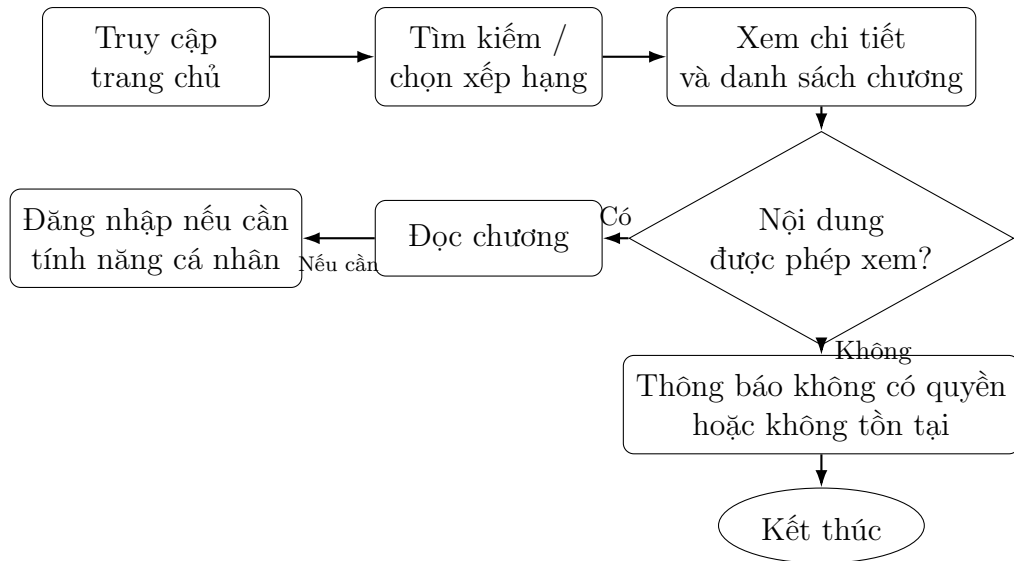
*** User và Moderator cần đăng nhập, có đủ Tinh thạch và mở khóa chương; Uploader và Admin được backend cho phép đọc trực tiếp theo logic kiểm tra quyền hiện tại.

Ma trận được rút ra trực tiếp từ router và middleware: tạo truyện dành cho Uploader, duyệt truyện dành cho Moderator/Admin, còn khu vực `/api/admin` chỉ dành cho Admin.

2.2. Luồng của khách và người dùng

2.2.1. Khám phá và đọc truyện khi chưa đăng nhập

Guest có thể duyệt, tìm kiếm, xem chi tiết và đọc nội dung công khai. Route chi tiết dùng `optionalAuth` để bổ sung dữ liệu cá nhân hóa khi request có token hợp lệ.

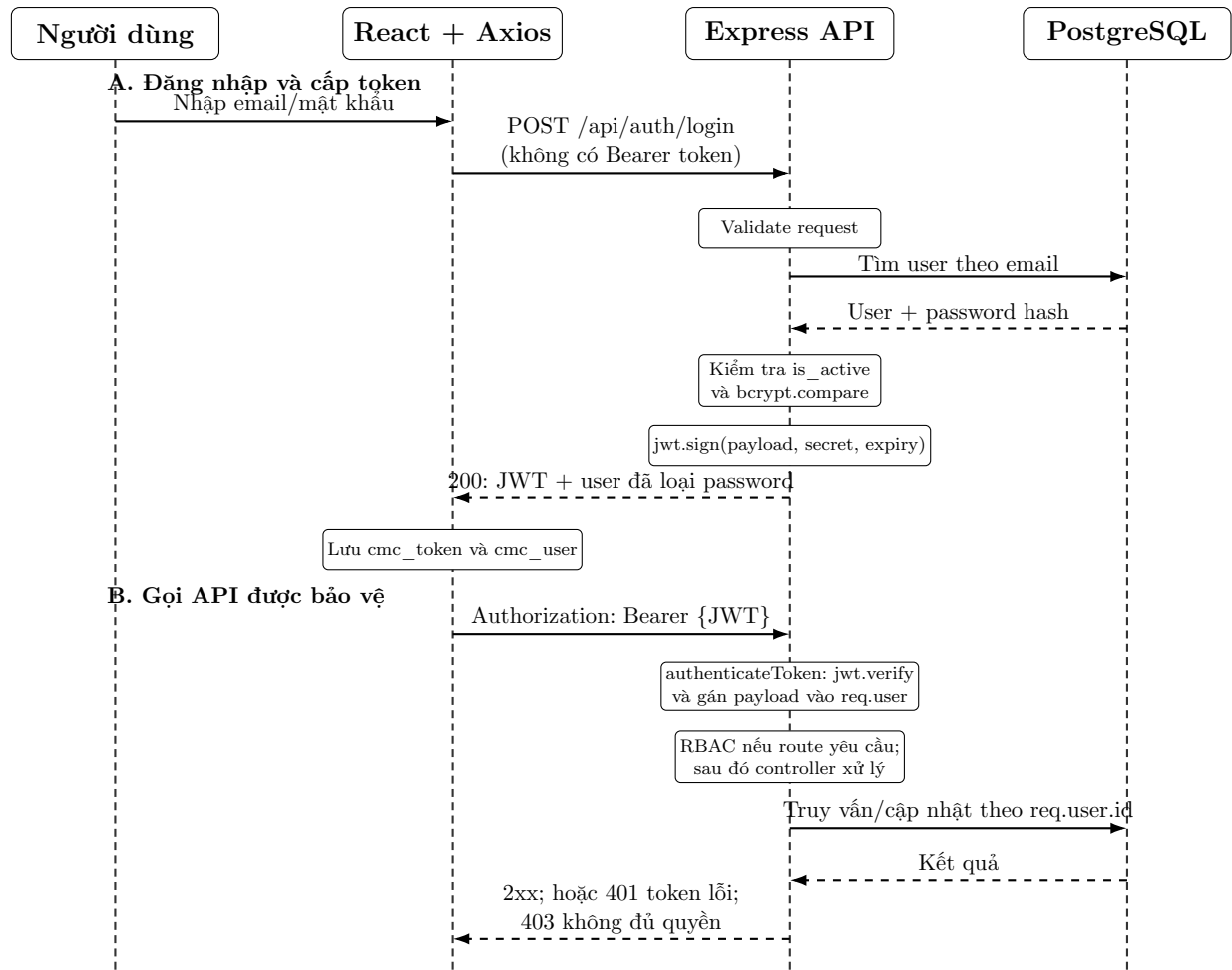


Hình 2.1: Luồng khám phá và đọc truyện của Guest

Giao diện cần phân biệt trạng thái đang tải, không có dữ liệu, không có quyền và lỗi mạng.

2.2.2. Đăng ký, đăng nhập và duy trì phiên

Frontend gửi thông tin đăng nhập đến POST/api/auth/login; backend kiểm tra mật khẩu và trả JWT. Token được lưu với key cmc_token, gắn vào request bằng Axios interceptor và xác minh tại middleware backend.



Hình 2.2: Trình tự xác thực và sử dụng JWT

Token thiếu, sai chữ ký hoặc hết hạn nhận mã 401; mã 403 được dùng khi tài khoản bị khóa hoặc người dùng đã xác thực nhưng không đủ vai trò. Response interceptor hiện xóa `cmc_token`, `cmc_user` và chuyển về trang đăng nhập khi nhận 401 hoặc 403, ngoại trừ mã `CHAPTER_LOCKED`. Luồng quên mật khẩu gồm yêu cầu OTP, xác minh và đặt mật khẩu mới; việc gửi email thật cần kiểm thử tích hợp riêng.

2.2.3. Theo dõi, lịch sử và tương tác

Sau khi đăng nhập, người dùng có thể theo dõi, đánh giá và bình luận. Một số thao tác dùng optimistic update và phải rollback nếu API thất bại.

Khi đọc chương, hệ thống gửi tiến độ đến POST/api/reading-history. Backend dùng định danh user từ JWT thay vì tin vào user id do client tự khai báo. Các endpoint lịch sử và chương đã đọc đều yêu cầu xác thực. Luồng diễn hình gồm:

- Người dùng mở trang chi tiết và chọn chương;
- Frontend lấy nội dung chương theo slug và số chương;
- Trình đọc hiển thị nội dung, tiến độ cuộn và tùy chọn đọc;
- Sau một mốc phù hợp, frontend lưu chương/vị trí/thời gian đọc;
- Lần truy cập sau, lịch sử cho phép quay lại truyện và chương gần nhất;
- Nếu request lưu thất bại, giao diện không được khẳng định đã đồng bộ.

2.2.4. Mở khóa chương trả phí bằng Tinh thạch

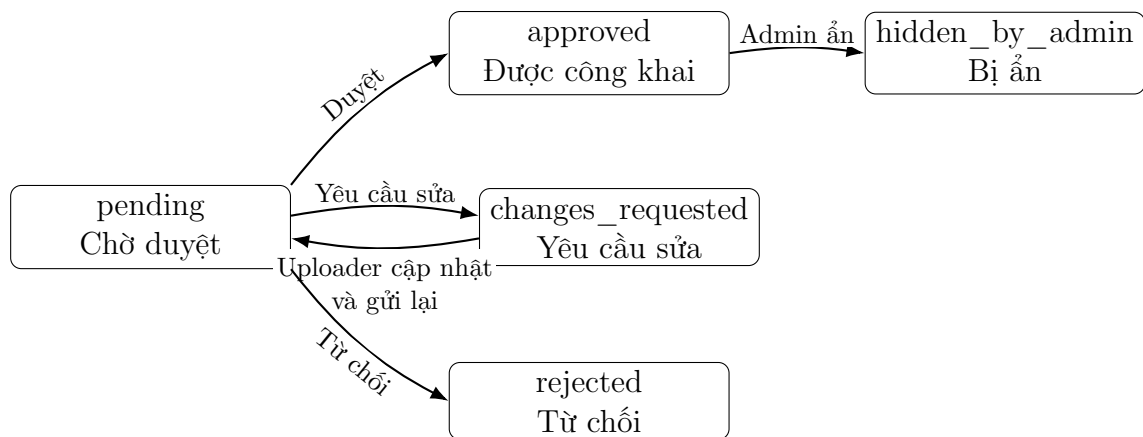
Với chương có trạng thái is_paid, Guest chỉ nhận được thông báo khóa và phần xem trước nội dung. Người dùng đã đăng nhập có thể chọn mở khóa bằng Tinh thạch; frontend gửi request POST/api/chapters/<id>/unlock. Backend kiểm tra trạng thái chương, quyền đọc và số dư trước khi thực hiện giao dịch. Khi thành công, hệ thống ghi nhận chương đã mở khóa, trừ 2 Tinh thạch khỏi ví và trả về số dư mới để giao diện cập nhật. Nếu số dư không đủ, API trả mã INSUFFICIENT_CRYSTALS; nếu chương đã mở khóa trước đó, hệ thống không trừ Tinh thạch lần hai. Report API áp dụng rate limiter; Moderator/Admin xử lý báo cáo và lưu kết quả giải quyết.

2.3. Luồng của Uploader

2.3.1. Tạo truyện và nhập nội dung

Uploader vào Dashboard, nhập metadata hoặc import tệp. Backend nhận thông tin, tạo truyện với `moderation_status='pending'` và mặc định chưa công khai. Khi import, hệ thống có bước preview để người dùng kiểm tra cách tách chương trước khi ghi dữ liệu. Source hỗ trợ tệp văn bản và EPUB; test kiểm tra thứ tự spine, tiêu đề chương, trường hợp không có tệp và fallback từ tên file.

Luồng tạo nội dung được thiết kế theo nguyên tắc “người đăng không tự duyệt nội dung của chính mình”. Test backend xác nhận Uploader không thể thêm chương trước khi truyện được Moderator duyệt và không thể dùng thao tác visibility để tự xuất bản. Điều này giảm rủi ro bỏ qua quy trình kiểm duyệt.



Hình 2.3: Vòng đời kiểm duyệt của truyện

2.3.2. Quản lý chương và cộng tác viên

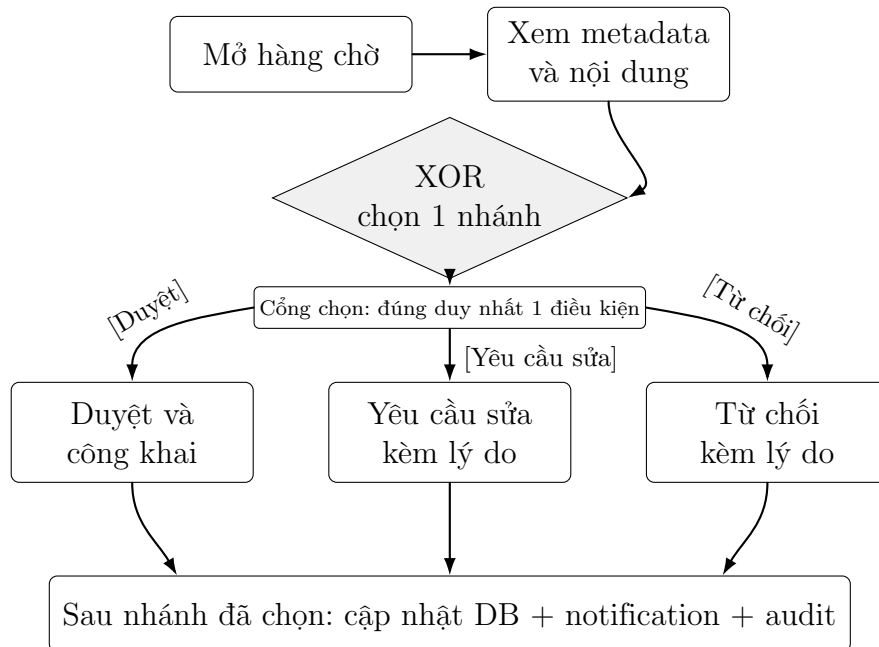
Sau khi truyện được duyệt, Uploader có thể thêm, sửa, xóa hoặc import chương theo quyền sở hữu/cộng tác. Backend phải kiểm tra quan hệ với truyện, không chỉ kiểm tra role chung. Ràng buộc `UNIQUE(story_id, chapter_number)` ngăn trùng số chương; controller chuyển xung đột dữ liệu thành phản hồi phù hợp thay vì trả lỗi máy chủ chung chung.

Chức năng cộng tác viên cho phép chia sẻ quyền làm nội dung trên một truyện. Đây là luồng nhạy cảm vì việc thêm hoặc xóa cộng tác viên thay đổi quyền truy cập gián tiếp. Khi hoàn thiện sản phẩm, cần kiểm tra đầy đủ: chỉ owner/admin được quản lý cộng tác viên, không tự thêm trùng, không cấp quyền vượt quá phạm vi truyện và phải có audit nếu yêu cầu vận hành đặt ra.

2.4. Luồng của Moderator

Moderator đăng nhập vào layout riêng, xem dashboard và hàng chờ truyện. Với mỗi truyện pending, người kiểm duyệt xem metadata và phần nội dung cần thiết trước khi chọn một trong ba hướng: duyệt, yêu cầu sửa kèm lý do, hoặc từ chối. Khi trạng thái thay đổi, backend cập nhật `reviewed_by`, `reviewed_at`, ghi audit và tạo notification phù hợp cho Uploader.

Với bình luận và hồ sơ bị báo cáo, Moderator dùng bộ lọc để tìm trường hợp cần xử lý, xem ngữ cảnh và cập nhật trạng thái. Test hiện có kiểm tra việc lọc ở server, số đếm trạng thái toàn cục và một số thao tác xử lý avatar. Điểm quan trọng là người kiểm duyệt cần thấy đủ ngữ cảnh để ra quyết định, nhưng không được nhận dữ liệu nhạy cảm không liên quan.



Hình 2.4: Luồng Moderator xử lý truyện chờ duyệt

2.5. Luồng của Admin

Admin có phạm vi vận hành rộng nhất nhưng vẫn đi qua xác thực, RBAC và audit. Các nhóm công việc chính gồm:

- Xem thống kê tổng quan và danh sách truyện;
- Tìm kiếm user, cập nhật role hoặc trạng thái hoạt động;
- Ẩn/hiện truyện, xóa bình luận theo quy định;
- Quản lý danh sách từ khóa và tier kiểm duyệt;
- Xem, phân loại và giải quyết báo cáo;
- Truy vấn audit log để đối chiếu hành động quản trị;
- Thực hiện các chức năng moderator khi cần.

Bảng 2.2: Luồng quản trị chính và dấu vết cần lưu

Thao tác	Kiểm tra trước khi thực hiện	Dấu vết sau thao tác
Đổi role user	Admin hợp lệ; role đích thuộc tập cho phép; tránh tự khóa quyền quản trị cuối cùng	Actor, user bị tác động, role cũ/mới, thời điểm.
Khóa/mở tài khoản	Xác định đúng user và lý do vận hành	Actor, trạng thái cũ/mới, user bị tác động.
Ẩn/hiện truyện	Kiểm tra truyện tồn tại và quyền quản trị	Action, story id, trạng thái hiển thị.
Quản lý từ khóa	Keyword/tier hợp lệ; tránh bản ghi trùng	Tạo/sửa/xóa từ khóa và tier; không ghi dữ liệu nhạy cảm.
Xử lý báo cáo	Report còn tồn tại; action phù hợp target	Trạng thái, action, note, resolver và thời điểm.
Xóa bình luận	Xác định vi phạm và phạm vi tác động	Actor, comment id và kết quả HTTP thành công.

Audit middleware chỉ ghi mutation quản trị thành công và làm sạch trường nhạy cảm trong request. Test kiểm tra password/credential lồng nhau không bị đưa vào details. Đây là yêu cầu bảo mật quan trọng: audit log phục vụ truy vết nhưng không được trở thành nơi rò rỉ bí mật.

2.6. Luồng báo cáo vi phạm

Luồng report kết nối người dùng, moderator và admin. Người dùng đăng nhập chọn đối tượng cần báo cáo, lý do và mô tả. Rate limiter hạn chế việc gửi lặp; controller xác thực target và tạo report. Moderator/Admin lọc theo trạng thái, xem ngữ cảnh, chọn hành động giải quyết rồi cập nhật trạng thái cùng ghi chú.

Các test backend xác nhận một số quy tắc: tạo report truyện hợp lệ; gắn báo cáo avatar với tác giả bình luận liên quan; ngăn spam trong một giờ; lọc report theo trạng thái; trả 404 khi report không tồn tại; không dùng action của chapter cho report comment; và lưu dữ liệu giải quyết. Những test này chứng minh logic nhánh quan trọng, nhưng vẫn cần demo trực quan để xác nhận UI truyền đúng payload và hiển thị phản hồi dễ hiểu.

2.7. Truy vết luồng đến endpoint và source

Bảng 2.3: Truy vết luồng nghiệp vụ đến API và module hiện thực

Luồng	Endpoint tiêu biểu	Quyền	Module chính
Đăng nhập	POST/api/auth/login	Public	authRoutes, authController
Xem truyện	GET/api/stories/by-slug/:slug	Optional auth	storyRoutes, storyController
Đọc chương	GET/api/stories/by-slug/:storySlug/chapters/:chapterNumber	Optional auth	chapterController
Lưu tiến độ	POST/api/reading-history	Authenticated	readingHistory controller
Theo dõi	POST/DELETE/api/follows/:storyId	Authenticated	followRoutes, model liên quan
Tạo truyện	POST/api/stories	Uploader	storyController.createStory
Import truyện	POST/api/stories/import-file	Uploader	upload middle-ware, import service
Duyệt truyện	PATCH/api/moderator/pending-stories/:id/approve	Moderator/Admin	moderator controller, audit middleware
Xử lý report	PATCH/api/reports/:id/process	Moderator/Admin	report controller, audit middleware
Đổi role user	PATCH/api/admin/users/:id/role	Admin	admin controller, audit middleware
Quản lý từ khóa	/api/admin/bad-words	Admin	bad word controller/model
Xem audit	GET/api/audit-logs	Admin/Moderator theo phạm vi	audit log route, controller, model

2.8. Quy tắc xử lý lỗi và an toàn luồng

Một luồng được xem là hoàn chỉnh khi có cả đường thành công và đường lỗi. Các nguyên tắc được áp dụng hoặc cần duy trì gồm:

- **400** cho dữ liệu đầu vào không hợp lệ, kèm thông báo đủ để người dùng sửa;
- **401** khi thiếu hoặc sai phiên xác thực;
- **403** khi đã xác thực nhưng không đủ quyền;
- **404** khi tài nguyên không tồn tại hoặc không nên lộ sự tồn tại;
- **409** cho xung đột nghiệp vụ như trùng số chương;
- **429** khi vượt giới hạn request/report;
- **500** chỉ cho lỗi ngoài dự kiến, không trả stack trace hoặc secret ra client;
- Mutation nhiều bước cần transaction cùng một DB client để bảo đảm rollback nhất quán;
- Optimistic UI phải hoàn tác khi API thất bại;
- Mọi quyết định quyền phải được lặp lại ở backend dù frontend đã ấn nút.

2.9. Kết luận chương

Chương 2 đã mô tả luồng của Guest, User, Uploader, Moderator và Admin, gồm trạng thái, ngoại lệ và điểm kiểm soát quyền. Các luồng được truy vết tới endpoint để làm cơ sở cho phần triển khai.

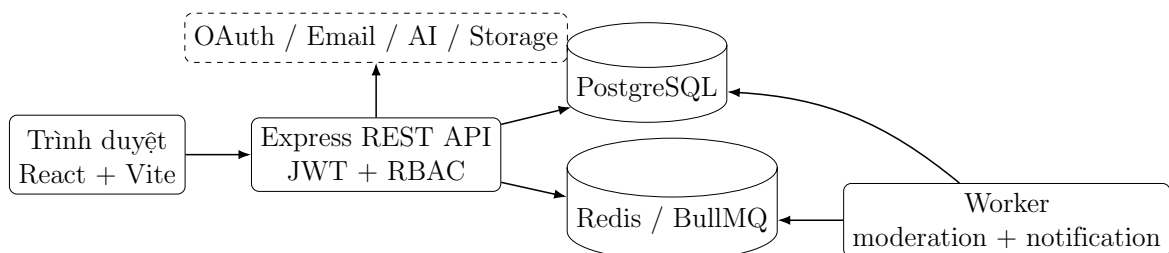
Chương 3. TRIỂN KHAI HỆ THỐNG

3.1. Kiến trúc phần mềm tổng thể

3.1.1. Lựa chọn mô hình kiến trúc

Mã nguồn hiện thực kiến trúc frontend/backend tách rời. Frontend là ứng dụng đơn trang (SPA) xây dựng bằng React và Vite; trình duyệt trao đổi với REST API Express qua HTTP/JSON. Backend xác thực JWT, điều phối nghiệp vụ và truy vấn PostgreSQL qua connection pool. Redis/BullMQ phục vụ hàng đợi; worker chạy ở tiến trình riêng để xử lý kiểm duyệt và thông báo. Một số dịch vụ bên ngoài được tích hợp cho OAuth, email, lưu trữ và AI [2, 3, 4].

Frontend đảm nhiệm giao diện, tương tác và điều hướng phía client. Backend được tổ chức theo các lớp Route–Controller–Model/Service; cách tách này giúp định tuyến, xử lý nghiệp vụ và truy cập dữ liệu có trách nhiệm rõ ràng [5].



Hình 3.1: Kiến trúc logic của hệ thống theo source hiện tại

Trong request thông thường, React gọi API; middleware kiểm tra CORS, parse body, xác thực và phân quyền; controller validate/điều phối; model hoặc service truy cập dữ liệu; error handler chuẩn hóa phản hồi. Với tác vụ nền, API ghi job vào queue và trả phản hồi sớm; worker nhận job, xử lý rồi cập nhật PostgreSQL hoặc tạo notification. Việc tách process giúp tránh chặn event loop của web server, nhưng chỉ phát huy tác dụng khi Redis và worker được cấu hình/giám sát đúng.

3.1.2. Công nghệ và trách nhiệm

Bảng 3.1: Stack công nghệ được xác nhận từ package và cấu hình dự án

Lớp	Công nghệ	Trách nhiệm
Frontend	React 18, React Router 6, Vite 5	Component UI, route SPA, state phía client và build tài nguyên tĩnh.
Giao tiếp	Axios	Base URL, JSON, gắn JWT và xử lý lỗi 401/403 tập trung.
Trình bày	CSS, Bootstrap 5, Tailwind CSS trong dependencies	Bố cục, responsive, trạng thái sáng/tối và trải nghiệm đọc.
Backend	Node.js, Express 4	REST API, middleware pipeline, controller và tích hợp dịch vụ.
Xác thực	JWT, bcryptjs, Google Auth Library	Phiên không trạng thái, băm mật khẩu và Google OAuth.
Cơ sở dữ liệu	PostgreSQL, thư viện pg	Dữ liệu quan hệ, ràng buộc, index và connection pool.
Tác vụ nền	Redis, BullMQ	Queue cho moderation/notification và worker riêng.
Lưu trữ/tích hợp	Supabase, Resend, Groq/Gemini	Ảnh, email OTP và tính năng AI qua backend.
Kiểm thử	Jest, Supertest, Vitest, Testing Library, Playwright	Unit/integration mức module và kịch bản E2E theo cấu hình.
Triển khai	Vercel-compatible SPA, Render web service/worker	Hosting frontend tĩnh, API và process worker.

3.2. Triển khai frontend

3.2.1. Cấu trúc thư mục

Entry point frontend là frontend/src/main.jsx, sau đó render App.jsx. Mã nguồn được tổ chức mô-đun hóa rõ ràng theo chuẩn ứng dụng React/Express với các thư mục pages, components, layouts, contexts, services, utils, styles và data. Thư mục pages gồm các component trang chính và thư mục components chứa các thành phần tái sử dụng.



Hình 3.2: Cấu trúc thư mục frontend

Bảng 3.2: Trách nhiệm các thư mục frontend chính

Thư mục/tệp	Trách nhiệm
App.jsx	Khai báo route, layout công khai, Admin, Moderator và route guard.
pages/	Component cấp trang cho Home, Search, Reader, Account, Dashboard và quản trị.
components/	Thành phần tái sử dụng như Navbar, StoryCard, CommentSection, FollowButton.
layouts/	Khung giao diện riêng cho Admin và Moderator.
contexts/	Trạng thái xác thực và theme chia sẻ trong cây component.
services/ \protect\ penalty\z@ {api.js}	Axios client và facade endpoint theo domain.
utils/	Validation, slug, số chương và chuẩn hóa lỗi.
styles/	CSS chung, dark mode, reader và màn hình quản lý.

3.2.2. Điều hướng và bảo vệ route

App.jsx chia route thành ba nhóm. Nhóm user/public dùng Navbar và Footer; nhóm Admin dùng layout riêng (AdminLayout); nhóm Moderator dùng layout riêng (ModeratorLayout). ProtectedRoute yêu cầu đăng nhập,

còn `RoleProtectedRoute` kiểm tra role trước khi render Dashboard hoặc layout quản trị. Cấu trúc này giảm lặp và giúp sidebar/menu nhất quán trong từng khu vực.

```

frontend/src/App.jsx
119     <Route
120       path="/dashboard"
121       element={
122         <RoleProtectedRoute allowedRoles={['Uploader', 'Admin']}>
123           <DashboardPage />
124         </RoleProtectedRoute>
125       }
126     />
131     /* ADMIN LAYOUT RIÊNG */
134     <RoleProtectedRoute allowedRoles={['Admin']}>
135       <AdminLayout />
136     </RoleProtectedRoute>
149     /* MODERATOR LAYOUT RIÊNG */
152     <RoleProtectedRoute allowedRoles={['Moderator', 'Admin']}>
153       <ModeratorLayout />
154     </RoleProtectedRoute>

frontend/src/components/ProtectedRoute.jsx
5  function ProtectedRoute({ children }) {
6    const { isAuthenticated } = useAuth();
7    const location = useLocation();
9    if (!isAuthenticated) {
10     return <Navigate to="/login" replace state={{ from: location }} />;
11   }
13   return children;

frontend/src/components/RoleProtectedRoute.jsx
4  function RoleProtectedRoute({ allowedRoles = [], children }) {
6    const { isAuthenticated, user, loading } = useAuth();
7    const location = useLocation();
16   if (!isAuthenticated) {
17     return <Navigate to="/login" replace state={{ from: location }} />;
18   }
21   const userRole = user?.role?.toLowerCase();
22   const allowed = allowedRoles.map(r => r.toLowerCase());
24   if (allowedRoles.length > 0 && !allowed.includes(userRole)) {
25     return <Navigate to="/" replace />;
26   }
28   return children;
29 }

```

Hình 3.3: Đoạn code chính về route và route guard ở frontend

Route frontend không thay thế quyền backend. Nếu chỉ ẩn menu Admin nhưng API không có middleware, người dùng vẫn có thể gọi endpoint bằng công cụ khác. Source hiện đã đặt RBAC ở backend; frontend guard được xem là lớp trải nghiệm và phòng lỗi điều hướng.

3.2.3. Lớp gọi API và quản lý phiên

`Services/api.js` tạo một Axios instance có base URL từ biến môi trường, fallback về local khi phát triển. Request interceptor tự động đính kèm JWT Bearer token vào header; response interceptor xử lý tập trung các mã lỗi xác thực (401/403), thực hiện xóa phiên làm việc và điều hướng về trang đăng nhập. Các endpoint được mô-đun hóa theo domain nghiệp vụ như auth, stories, chapters, comments, follows, reports, admin và moderator.

Ưu điểm của facade tập trung là giúp các component giao diện gọi API ngắn gọn, tránh lặp lại cấu hình header và base URL.

Base URL theo biến môi trường	Dòng 15–21
<pre> 15 function getBaseUrl() { 16 return (17 import.meta.env.VITE_API_URL // Vite environment variable 18 import.meta.env.REACT_APP_API_URL // Create React App (legacy) 19 'http://localhost:5000/api' 20); 21 } </pre>	
Axios instance dùng chung	Dòng 37–43
<pre> 37 const apiClient = axios.create({ 38 baseUrl: getBaseUrl(), 39 headers: { 40 'Content-Type': 'application/json', 41 Accept: 'application/json', 42 }, 43 }); </pre>	
Request interceptor — tự động gắn Bearer token	Dòng 50–64
<pre> 50 apiClient.interceptors.request.use((config) => { 51 try { 52 const token = localStorage.getItem('cmc_token'); 53 if (token && token !== 'null') { 54 config.headers['Authorization'] = `Bearer \${token}`; 55 } 56 } catch (err) { 57 // localStorage may be unavailable in some environments (private mode, strict policies). 58 // Silently ignore so the app doesn't spam the console or throw runtime errors. 59 } 60 61 return config; 62 }, (error) => { 63 return Promise.reject(error); 64 }); </pre>	
Response interceptor — xử lý lỗi 401/403	Dòng 76–91
<pre> 76 apiClient.interceptors.response.use(77 (response) => response, // Nếu thành công, trả về nguyên response 78 (error) => { 79 if ((error?.response?.status === 401 error?.response?.status === 403) && typeof window !== 'undefined') { 80 const path = window.location.pathname; 81 82 // Chỉ redirect nếu KHÔNG đang ở các trang public (login, register, home) 83 // Tránh redirect loop: login page nhận 401/403 → redirect về login 84 if (!path.startsWith('/login') && !path.startsWith('/register') && path !== '/') { 85 clearAuthStorage(); 86 window.location.href = '/login'; 87 } 88 } 89 return Promise.reject(error); 90 } 91); </pre>	

Hình 3.4: Đoạn code chính về Axios interceptor ở frontend

3.2.4. Mở khóa chương ở frontend

Facade API khai báo `API.chapters.unlock(chapterId)` để gọi endpoint mở khóa. Tại `StoryDetailPage`, chương chưa có quyền đọc được gắn nhãn khóa và người dùng xác nhận trước khi gửi request. Tại `ChapterReaderPage`, response `CHAPTER_LOCKED` được chuyển thành giao diện xem trước thay vì hiển thị toàn bộ nội dung. Sau khi API mở khóa thành công, `setCrystalBalance` cập nhật số dư trong context đăng nhập và trang đọc được tải lại để lấy nội dung đầy đủ.

Tùy chọn `auto_unlock_next_chapter` được lưu trong cấu hình cá nhân. Nhằm bảo vệ số dư Tinh thạch của người dùng và tránh việc tự động trừ chi phí ngoài ý muốn khi cuộn qua chương mới, tính năng mở khóa chương tiếp theo được thiết kế ở dạng xác nhận thủ công rõ ràng trên giao diện.

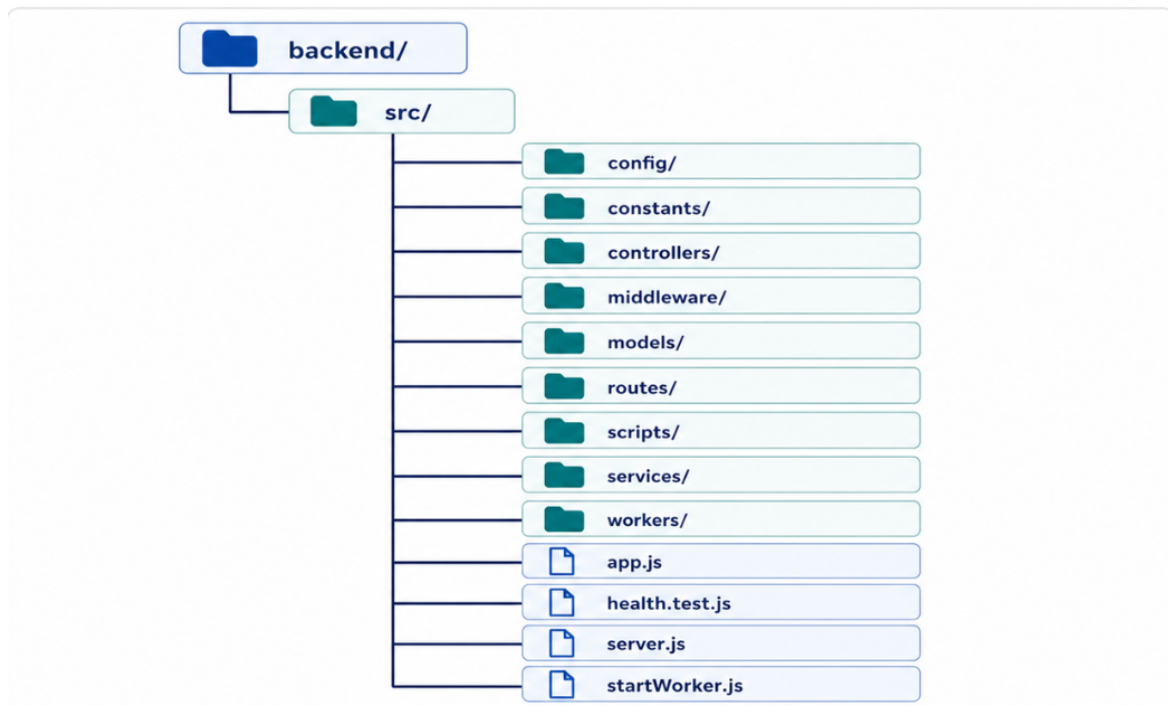
3.2.5. Trạng thái giao diện và trải nghiệm

Hệ thống tích hợp skeleton screen trong thời gian chờ tải dữ liệu và cơ chế optimistic UI kèm rollback tự động khi thao tác thất bại (như follow, đánh giá sao, bình chọn). Theme và tùy chọn hiển thị được quản lý qua React Context. Giao diện được tối ưu đồng bộ cho cả 4 trạng thái: đang tải (loading), thành công có dữ liệu, danh sách rỗng và thông báo lỗi mạng.

3.3. Triển khai backend

3.3.1. Cấu trúc phân lớp

Backend bắt đầu ở `src/server.js`, tạo HTTP server từ `app.js`. Router chỉ định URL và middleware; controller validate, điều phối và tạo HTTP response; model/service thực hiện query hoặc tích hợp bên ngoài; worker xử lý job. Thống kê source chính cho thấy 16 file route, 17 controller, 17 model, 9 service và 6 middleware, không tính file test.



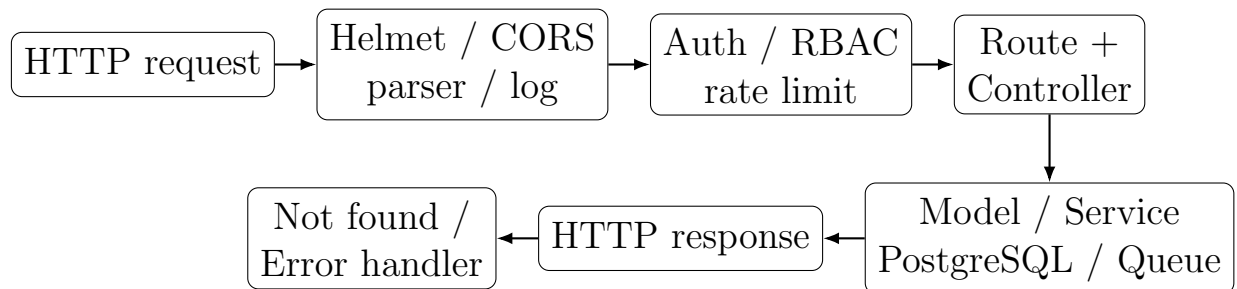
Hình 3.5: Cấu trúc thư mục backend

Bảng 3.3: Phân lớp backend và nguyên tắc phụ thuộc

Lớp	Trách nhiệm	Nguyên tắc
Routes	URL, method, auth, role, upload, rate limit, audit	Không chứa truy vấn SQL dài hoặc nghiệp vụ phức tạp.
Controllers	Validate request, điều phối model/service, response	Không tin user id/role do client tự gửi khi đã có JWT.
Models	Truy vấn và ánh xạ dữ liệu	Parameterized query; transaction dùng cùng DB client.
Services	Nghiệp vụ/tích hợp dùng lại	Che chi tiết AI, email, storage, queue khỏi controller.
Middleware	Xử lý cắt ngang	Auth trước role; audit sau khi biết response thành công.
Workers	Consumer job nền	Chạy process riêng, idempotent khi có thể.
Config	Biến môi trường và client hạ tầng	Secret chỉ ở backend; không commit .env.

3.3.2. Pipeline request

App.js lần lượt cấu hình Helmet, CORS, log HTTP, các parser JSON và URL-encoded, static cover, health check, 16 nhóm router rồi not-found và error handler. Thứ tự middleware quan trọng: error handler phải ở cuối; role middleware phải sau auth; audit cần quan sát kết quả response nhưng không được ghi dữ liệu nhạy cảm.



Hình 3.6: Pipeline xử lý request ở backend

```

backend/src/app.js
31  const app = express();
33  // Security & Middleware
34  app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
35  app.use(
36    cors({
37      credentials: true,
38    })
39  );
40
41  app.use(morgan(env.isDevelopment ? 'dev' : 'combined'));
42  app.use(express.json({ limit: '10mb' }));
43  app.use(express.urlencoded({ extended: true }));
44  app.use('/uploads/covers', express.static(uploadDir));
45
46  // Health Check
47  app.get('/health', (req, res) => {
48    res.status(200).json({
49      success: true,
50      message: 'CMC Truyen backend is running',
51      environment: env.NODE_ENV,
52    });
53  });
54
55  // Route đăng ký
56  app.use('/api/auth', authRoutes);
57  app.use('/api/stories', storyRoutes);
58  app.use('/api/reading-history', readingHistoryRoutes);
59  app.use('/api/chapters', chapterRoutes);
60  app.use('/api/ai', aiRoutes);
61  app.use('/api/comments', commentRoutes);
62  app.use('/api/follows', followRoutes);
63  app.use('/api/notifications', notificationRoutes);
64  app.use('/api/preferences', preferencesRoutes);
65  app.use('/api/upload', uploadRoutes);
66  app.use('/api/admin', adminRoutes);
67  app.use('/api/moderator', moderatorRoutes);
68  app.use('/api/tags', tagRoutes);
69  app.use('/api/reports', reportRoutes);
70  app.use('/api/rankings', rankingRoutes);
71  app.use('/api/audit-logs', auditLogRoutes);
72
73  // Error Handling
74  app.use(notFoundHandler);
75  app.use(errorHandler);
76
77  module.exports = app;
  
```

Hình 3.7: Đoạn code chính cấu hình middleware và mount router

3.3.3. Xác thực, phân quyền và audit

AuthenticateToken đọc header Authorization, yêu cầu định dạng Bearer, xác minh JWT bằng secret từ environment và gán payload vào req.user. optionalAuth không từ chối Guest; token lỗi được xem như không có phiên ở các route công khai. authorizeRole chuẩn hóa chữ thường để tránh sai khác giữa Admin và admin. Cấu trúc và cách kiểm tra token được đối chiếu với đặc tả JWT [7].

Audit middleware bao quanh các thao tác làm thay đổi dữ liệu quản trị như đổi vai trò, đổi trạng thái người dùng, duyệt truyện, xử lý báo cáo và quản lý từ khóa. Kiểm thử xác nhận hệ thống chỉ ghi hành động thành công, giới hạn phạm vi Moderator và loại dữ liệu mật khẩu/credential khỏi nhật ký. Các biện pháp xác thực, phân quyền, kiểm soát đầu vào và ghi log được rà soát theo nhóm rủi ro ứng dụng web phổ biến của OWASP [10].

```

backend/src/middleware/authMiddleware.js
16 function authenticateToken(req, res, next) {
17   const authHeader = req.headers.authorization;
22   if (!authHeader || !authHeader.startsWith('Bearer ')) {
23     return res.status(401).json({
24       success: false,
25       message: 'No token provided',
26     });
30   const token = authHeader.split(' ')[1];
32   try {
35     const decoded = jwt.verify(token, env.JWT_SECRET);
39     req.user = decoded;
40     return next();
41   } catch (error) {
43     return res.status(401).json({
44       success: false,
45       message: 'Invalid or expired token',
46     });
48   }
60 function optionalAuth(req, res, next) {
61   const authHeader = req.headers.authorization;
64   if (authHeader && authHeader.startsWith('Bearer ')) {
65     const token = authHeader.split(' ')[1];
66     try {
68       req.user = jwt.verify(token, env.JWT_SECRET);
69     } catch {
71       req.user = null;
73     } else {
75       req.user = null;
78       return next();
79     }
}

backend/src/middleware/roleMiddleware.js
5 function authorizeRole(...roles) {
6   return (req, res, next) => {
8     if (!req.user || !req.user.role) {
9       return res.status(403).json({
11        message: 'Access denied: No role information found',
12      });
17      const userRole = req.user.role.toLowerCase();
18      const allowedRoles = roles.map(role => role.toLowerCase());
21      if (!allowedRoles.includes(userRole)) {
22        return res.status(403).json({
24          message: 'Access denied: Insufficient permissions',
25        });
29      return next();
30    };
31  }
}

```

Hình 3.8: Đoạn code chính về xác thực JWT và RBAC

3.3.4. Kiểm soát truy cập và mở khóa chương trả phí

Service chapterAccessService quyết định quyền đọc chương. Chương miễn phí được đọc công khai; chương trả phí yêu cầu người dùng đăng nhập và có bản ghi trong chapter_unlocks. Admin, Uploader, tác giả truyện và cộng tác viên được xử lý theo nhánh quyền đặc biệt của backend. Khi chưa đủ quyền, API không trả toàn bộ nội dung mà trả response CHAPTER_LOCKED kèm phần xem trước, chi phí mở khóa và số dư hiện có.

Route POST/api/chapters/<id>/unlock yêu cầu JWT. Controller gọi Wallet.unlockChapter; model sử dụng transaction BEGIN/COMMIT/ROLLBACK và khóa dòng số dư bằng FORUPDATE. Trong cùng transaction, hệ thống kiểm tra chương đã mở khóa hay chưa, kiểm tra số dư, cập nhật users.crystal_balance, tạo bản ghi chapter_unlocks và ghi lịch sử vào crystal_transactions. Ràng buộc duy nhất theo cặp người dùng-chương kết hợp với transaction giúp ngăn trừ Tinh thạch hai lần khi request được gửi đồng thời.

3.3.5. Tác vụ nền và dịch vụ AI

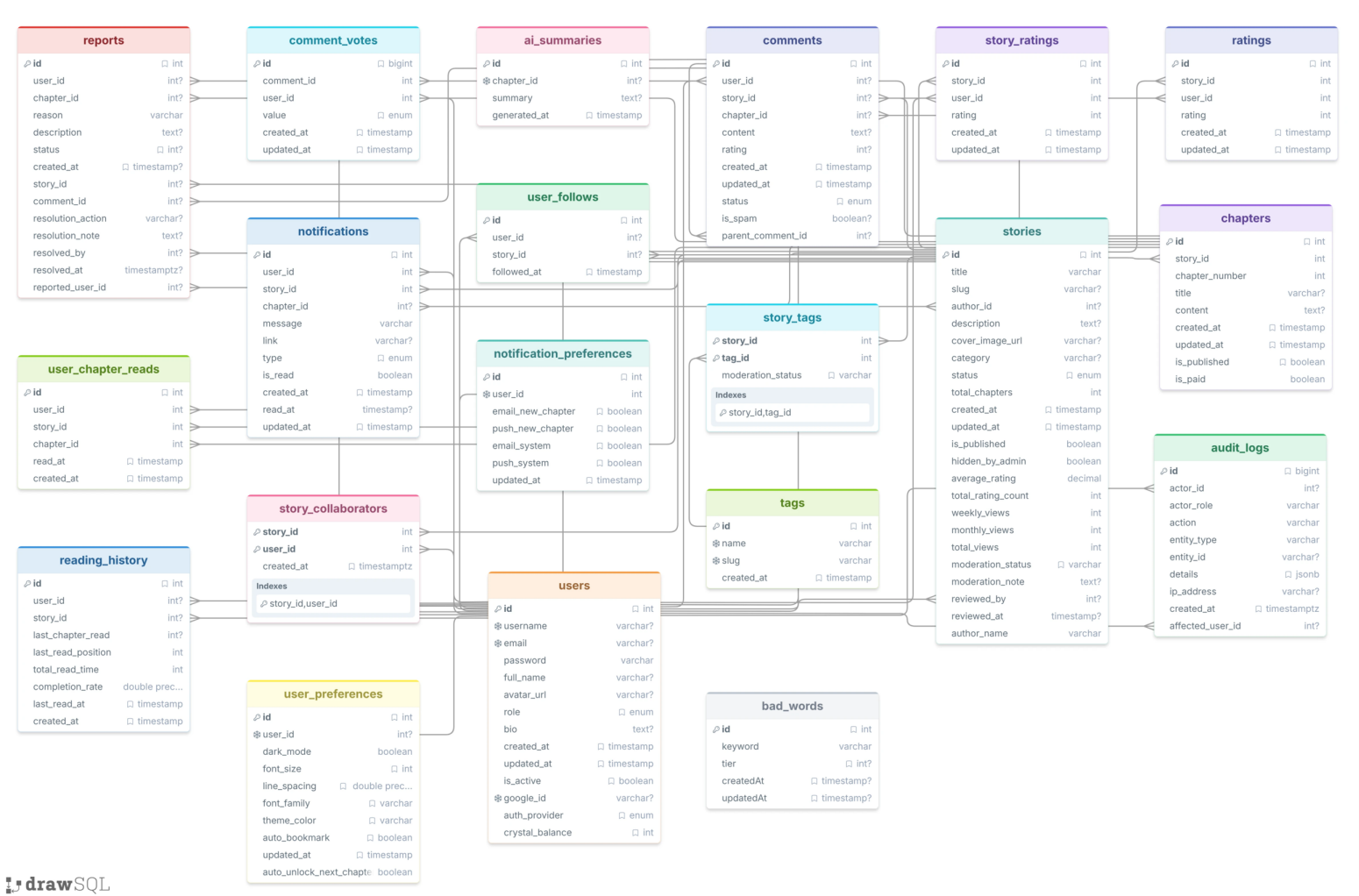
StartWorker.js khởi động moderation worker và notification worker. Queue dùng REDIS_URL; API và worker phải trỏ cùng Redis [8, 9]. Với AI, backend ưu tiên Groq rồi fallback Gemini, có timeout và cache; tóm tắt chương được lưu ở ai_summaries. API key không đi xuống frontend vì mã phía client có thể được người dùng xem trực tiếp.

Tuy nhiên, fallback giữa hai mô hình có thể tạo kết quả khác nhau; hệ thống cần log provider, thời gian và lỗi ở mức đủ chẩn đoán nhưng không ghi prompt nhạy cảm. Với moderation tự động, quyết định có tác động lớn nên có cơ chế review của người vận hành thay vì coi output AI là phán quyết tuyệt đối.

3.4. Triển khai cơ sở dữ liệu

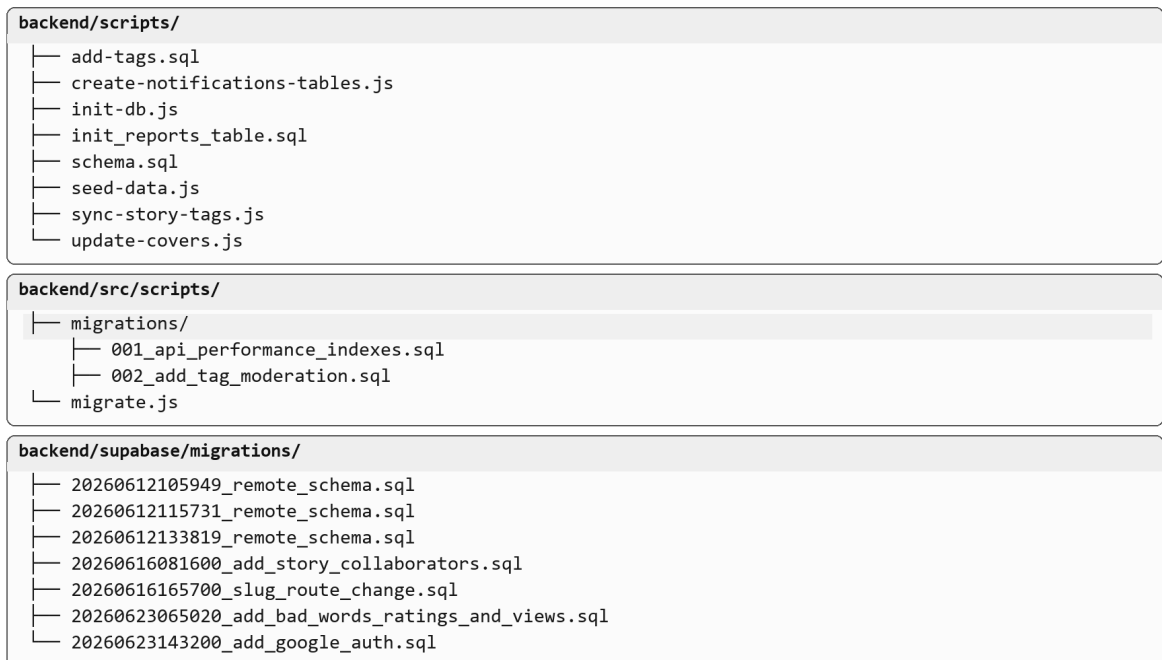
3.4.1. Mô hình dữ liệu

Schema chính backend/scripts/schema.sql định nghĩa các nhóm dữ liệu về tài khoản, truyện và chương, lịch sử đọc, tương tác, thông báo, kiểm duyệt và nhật ký quản trị. Các script riêng bổ sung tag, liên kết truyện-tag và báo cáo vi phạm. Thiết kế sử dụng khóa chính, khóa ngoại, UNIQUE và CHECK để bảo đảm tính toàn vẹn dữ liệu [6].



Hình 3.9: Lược đồ ERD của cơ sở dữ liệu CMC Truyen

Hình 3.9 thể hiện các quan hệ chính của hệ thống. users, stories và chapters là ba thực thể trung tâm; các bảng lịch sử đọc, theo dõi, đánh giá, bình luận và thông báo liên kết hành vi người dùng với nội dung. Nhóm bảng reports, bad_words và audit_logs phục vụ kiểm duyệt và truy vết.



Hình 3.10: Cấu trúc script, migration và schema cơ sở dữ liệu

01 • users

schema.sql • lines 21-29

PRIMARY KEY • UNIQUE • CHECK

```

21 CREATE TABLE IF NOT EXISTS users (
22     id SERIAL PRIMARY KEY,
23     username VARCHAR(100) UNIQUE,
24     email VARCHAR(255) UNIQUE,
25     password VARCHAR(255) NOT NULL,
26     full_name VARCHAR(255),
27     avatar_url VARCHAR(500),
28     role VARCHAR(50) NOT NULL DEFAULT 'User'
29     CHECK (role IN ('Admin', 'Uploader', 'User', 'Moderator')),

```

02 • chapters

schema.sql • lines 70-80

PRIMARY KEY • FOREIGN KEY • UNIQUE

```

70 CREATE TABLE IF NOT EXISTS chapters (
71     id SERIAL PRIMARY KEY,
72     story_id INTEGER NOT NULL REFERENCES stories(id) ON DELETE CASCADE,
73     chapter_number INTEGER NOT NULL,
74     title VARCHAR(255),
75     content TEXT,
76     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
77     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
78     is_published BOOLEAN NOT NULL DEFAULT true,
79     UNIQUE (story_id, chapter_number)
80 );

```

03 • story_collaborators

schema.sql • lines 199-204

COMPOSITE PRIMARY • FOREIGN KEYS

```

199 CREATE TABLE IF NOT EXISTS story_collaborators (
200     story_id INTEGER NOT NULL REFERENCES stories(id) ON DELETE CASCADE,
201     user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
202     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
203     PRIMARY KEY (story_id, user_id)
204 );

```

04 • indexes

schema.sql • lines 256-262

SIMPLE • PARTIAL • COMPOSITE

```

256 CREATE INDEX IF NOT EXISTS idx_stories_author_id ON stories(author_id);

257 CREATE INDEX IF NOT EXISTS idx_stories_slug ON stories(slug)
    WHERE slug IS NOT NULL;

260 CREATE INDEX IF NOT EXISTS idx_stories_category_published_created
    ON stories(category, created_at DESC) WHERE is_published = true;

262 CREATE INDEX IF NOT EXISTS idx_chapters_story_published_number
    ON chapters(story_id, chapter_number) WHERE is_published = true;

```

Hình 3.11: Đoạn SQL chính về bảng truyện, chương và ràng buộc

3.4.2. Connection pool, index và transaction

Backend cấu hình PostgreSQL Pool với tối đa 20 connection, idle timeout 30 giây và connection timeout 5 giây. Môi trường production có thể dùng DATABASE_URL; môi trường local có thể dùng các biến host/port/name/user/password. Index tập trung vào slug, danh sách truyện công khai, số chương, trạng thái bình luận, khóa ngoại và audit. API danh sách chương không trả trường content, qua đó giảm kích thước phản hồi.

Khi một nghiệp vụ có nhiều bước ghi dữ liệu, tất cả query từ BEGIN đến COMMIT/ROLLBACK phải dùng cùng client lấy từ pool.connect(). Nếu trộn pool.query() với client trong một transaction, các query có thể chạy trên connection khác và rollback không còn bảo đảm tính nguyên tử. Nghiệp vụ mở khóa chương là ví dụ điển hình cho transaction nhiều bước. Model Wallet.unlockChapter khóa bản ghi số dư của người dùng bằng FORUPDATE, kiểm tra bản ghi mở khóa hiện có và số dư trước khi ghi dữ liệu. Nếu bất kỳ bước nào thất bại, ROLLBACK bảo đảm số dư, lịch sử giao dịch và quyền đọc không rơi vào trạng thái không nhất quán.

3.5. Cấu hình và triển khai

3.5.1. Biến môi trường

Các biến chính gồm URL frontend/backend, PostgreSQL, Redis, JWT, Google OAuth, Resend, Supabase và nhà cung cấp AI. File .env không được commit. Frontend chỉ nhận biến public như VITE_API_URL; service key, JWT secret và AI key phải ở backend.

3.5.2. Topology triển khai

Render.yaml khai báo hai service Node: web service chạy backend và worker chạy src/startWorker.js. Frontend có vercel.json rewrite mọi route SPA về index.html. PostgreSQL và Redis không được khai báo trực tiếp trong file Render; chúng phải được cung cấp qua dịch vụ managed hoặc hạ tầng khác và cấu hình bằng environment.

Bảng 3.4: Nhóm biến môi trường và nguyên tắc bảo mật

Nhóm	Ví dụ tên biến	Nguyên tắc
Database	DATABASE_URL, DB_HOST, DB_NAME	Chỉ backend; production ưu tiên secret manager.
Auth	JWT_SECRET, GOOGLE_CLIENT_ID	Secret đủ mạnh; không dùng fallback production.
Queue	REDIS_URL	API và worker dùng cùng instance; bật TLS nếu nhà cung cấp yêu cầu.
Email/Storage	RESEND_API_KEY, SUPABASE_SERVICE_KEY	Service key không xuất hiện trong bundle frontend/log.
AI	GROQ_API_KEY, GEMINI_API_KEY	Chỉ backend; đặt timeout, quota và theo dõi lỗi.
Frontend	VITE_API_URL	Có thể công khai; trỏ đúng tiền tố /\protect\penalty\z@\}api.

Bảng 3.5: Liên kết bàn giao và minh chứng triển khai

Hạng mục	Liên kết
Website chạy thực tế	https://cmc-truyen.vercel.app
Kho source dùng để nộp	https://github.com/suzynotsusie/cmc-truyen-temp

3.6. Kiểm thử và kết quả xác minh

3.6.1. Chiến lược kiểm thử

Dự án áp dụng chiến lược kiểm thử theo mô hình tháp (Testing Pyramid) với ba tầng chính: unit/integration test chiếm phần lớn (Jest cho backend, Vitest và Testing Library cho frontend), E2E test tự động bằng Playwright cho các luồng nghiệp vụ quan trọng, và kiểm thử thủ công bổ sung cho kịch bản khó tự động hóa. Toàn bộ dependency ngoài (database, Redis, Supabase, email) được mock trong bộ test mặc định để đảm bảo CI ổn định; kiểm thử với hạ tầng thật được lên kế hoạch trên môi trường staging riêng. Bảng 3.6 tóm tắt công cụ theo từng tầng.

Bảng 3.6: Công cụ và phạm vi kiểm thử theo tầng

Tầng kiểm thử	Công cụ	Phạm vi
Unit / Integration	Jest, Supertest	Backend: controller, service, middleware, RBAC, audit
Component / Unit	Vitest, Testing Library	Frontend: component, hook, route guard, UI behavior
End-to-End	Playwright	Luồng đăng nhập, tìm kiếm và đọc truyện
Performance	Jest (budget test)	Parse 1.000 chương trong ngân sách thời gian định nghĩa
Thủ công	Trình duyệt, Dev-Tools	Responsive, edge case UX, luồng phân quyền phức tạp

3.6.2. Kết quả tổng hợp

Bộ test mặc định và build frontend được chạy trực tiếp trên workspace dự án. Kết quả được ghi ở Bảng 3.7; đây là số liệu của lần chạy thực tế, không phải con số giả định.

Bảng 3.7: Kết quả kiểm thử và build đã thực chạy

Hạng mục	Kết quả	Ghi chú quan sát
Backend (Jest)	121/121 pass	21 test suite pass; có log Redis ECONNREFUSED và Jest force-exit do BullMQ queue chưa đóng sau test.
Frontend (Vitest)	36/36 pass	11 test file pass; có cảnh báo tùy chọn esbuild/oxc deprecated từ toolchain.
Tổng test mặc định	157/157 pass	Lệnh root test hoàn thành exit code 0.
Frontend production build	Thành công	209 module transformed; build trong 2,13 giây; cảnh báo dynamic import cho mockStories.js.
Bundle JavaScript	641,96 kB	Gzip 192,47 kB; Vite cảnh báo chunk lớn hơn 500 kB.
E2E Playwright	6/6 pass	100% pass (6 kịch bản: login reader, login sai, mất mạng, login admin, tìm kiếm, đọc chương).
Performance (parse 1.000 chương)	Đạt ngân sách	Service parse tệp hoàn thành dưới ngưỡng thời gian định nghĩa trong storyFileImportService.\protect\penalty\z@{}performance.\protect\penalty\z@{}test.js.

Các bài kiểm thử đều đã vượt qua thành công, chứng minh các vùng logic cốt lõi đã được bảo vệ đầy đủ: từ health, auth, RBAC, audit, OTP, đến import file, ranking, kiểm duyệt, báo cáo và luồng xử lý truyện. Lỗi kết nối Redis ECONNREFUSED xuất hiện trong log là bình thường khi chạy ở môi trường local không có sẵn service Redis. Về việc Jest bị force-exit, nguyên nhân xuất phát từ việc hàng đợi BullMQ chưa được đóng đúng cách trong afterAll. Để khắc phục triệt để, chúng ta cần bổ sung logic teardown nhằm giải phóng kết nối, đồng thời tích hợp ioredis-mock để cô lập hoàn toàn các unit test. Cuối cùng, kích thước bundle JavaScript hiện đạt 641,96 kB, vượt quá giới hạn khuyến nghị 500 kB của Vite. Giải pháp cần triển khai là áp dụng route-level lazy loading kết hợp chia nhỏ vendor chunk, sau đó đo đạc lại để kiểm chứng.

3.6.3. Nhóm 1: Xác thực người dùng

Kết quả kiểm thử tự động cho luồng xác thực người dùng (đăng nhập, đăng ký, quên mật khẩu và xác thực OTP) được tổng hợp tại Bảng 3.8. Quá trình kiểm thử bao gồm 18 kịch bản, được thực thi qua các test suite: `authController.login.test.js`, `authController.account.test.js` và `otpService.test.js`, nhằm đảm bảo tính bảo mật và trải nghiệm xuyên suốt của hệ thống.

Bảng 3.8: Test case nhóm xác thực người dùng

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
1	Đăng nhập đúng tài khoản	Email <code>reader@example.com</code> , password <code>Password1!</code>	HTTP 200, JWT và user	Đăng nhập thành công; response không chứa password	HTTP 200, có token, password đã bị loại bỏ (Đạt)
2	Đúng email, sai mật khẩu	Email hợp lệ, password <code>WrongPassword1!</code>	HTTP 401, thông báo tiếng Việt	Không cấp token	HTTP 401, UI hiển thị thông báo tiếng Việt (Đạt)
3	Sai email, đúng mật khẩu của tài khoản khác	Email <code>unknown@example.com</code> , password hợp lệ	HTTP 401, cùng thông báo lỗi	Không đăng nhập và không tiết lộ email tồn tại hay không	HTTP 401, cùng thông báo tiếng Việt (Đạt)
4	Email đăng nhập sai định dạng	<code>reader-at-example.com</code>	HTTP 400, lỗi định dạng	Chặn trước khi truy vấn database	HTTP 400, lỗi hiển thị bằng tiếng Việt (Đạt)
5	Email dùng ký tự Unicode full-width	Chuỗi email toàn ký tự full-width	HTTP 400, dữ liệu không hợp lệ	Không chấp nhận định dạng ký tự sai	HTTP 400, không truy vấn user (Đạt)
6	Database lỗi khi đăng nhập	Credentials hợp lệ; DB không khả dụng	HTTP 500, thông báo chung	Không cấp token và không lộ lỗi nội bộ	HTTP 500, thông báo chung bằng tiếng Việt (Đạt)
7	Đăng ký hợp lệ	Username <code>Reader_01</code> , email hợp lệ, password mạnh	HTTP 201, token và role <code>User</code>	Tạo tài khoản, hash password, không trả hash	Role <code>User</code> , response không có password (Đạt)
8	Username đã tồn tại	Username trùng tài khoản hiện có	HTTP 409	Không kiểm tra/tạo tiếp bằng email	HTTP 409, không gọi tạo user (Đạt)
9	Email đã tồn tại	Username mới, email trùng	HTTP 409	Không tạo tài khoản trùng	HTTP 409, không gọi tạo user (Đạt)
Còn tiếp ở trang sau					

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
10	Username sai định dạng	Username có dấu hoặc khoảng trắng	HTTP 400	Chỉ nhận chữ không dấu, số, _, -	HTTP 400, thông báo tiếng Việt (Đạt)
11	Mật khẩu đăng ký yếu	password (thiếu chữ hoa, số, ký tự đặc biệt)	HTTP 400, thông báo tiêu chí	Yêu cầu tối thiểu 8 ký tự, chữ hoa, số và ký tự đặc biệt	HTTP 400, không tạo user (Đạt)
12	Chèn role qua request đăng ký	Payload có thêm role: Admin và is_active: false	User tạo với role User	Chống mass assignment	Field lạ bị loại bỏ, role vẫn là User (Đạt)
13	Quên mật khẩu với email tồn tại/không tồn tại	Hai email có trạng thái khác nhau	Cùng HTTP 200 và cùng message	Chống dò tìm email tài khoản	Hai response giống nhau (Đạt)
14	OTP sai hoặc hết hạn	OTP 000000 không hợp lệ	HTTP 400	Không cho reset password	HTTP 400, không cập nhật password (Đạt)
15	Reset khi chưa xác thực OTP	Password mới hợp lệ nhưng OTP chưa verified	HTTP 403	Bắt buộc xác thực OTP trước	HTTP 403, không cập nhật password (Đạt)
16	Reset password thành công	OTP verified; password mạnh và xác nhận khớp	HTTP 200	Hash password mới và xóa OTP đã dùng	Password được cập nhật, OTP keys được xóa (Đạt)
17	Gửi form login hai lần (double-click)	Double-click nút đăng nhập khi request đang xử lý	Chỉ một request; nút bị disabled	Không tạo request đăng nhập trùng	Hàm login chỉ được gọi một lần (Đạt)
18	Mất mạng khi đăng nhập	API login bị ngắt kết nối	UI giữ nguyên /login và hiện lỗi	Cho phép người dùng thử lại, không crash	Alert lỗi hiển thị, không chuyển trang (Đạt)

3.6.4. Nhóm 2: Upload truyện và kiểm duyệt

Quy trình đăng tải truyện và các bước kiểm duyệt được đánh giá qua 21 kịch bản kiểm thử tự động (Bảng 3.9). Quá trình này xác minh các ràng buộc về quyền sở hữu, giới hạn kích thước tệp tải lên (tối đa 25 MB) và luồng công việc xét duyệt của Moderator. Các test suite đảm nhiệm bao gồm: `uploadStoryFile.test.js`, `chapterController.upload.test.js`, `storyWorkflow.test.js` và `moderatorController.test.js`.

Bảng 3.9: Test case nhóm upload truyện và kiểm duyệt

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
1	Upload TXT UTF-8 vào truyện đã duyệt	Uploader là chủ sở hữu; <code>truyen.txt</code> UTF-8; truyện approved	HTTP 201, <code>imported_count: 1</code>	Tạo đúng chương và số chương kế tiếp	Tạo 1 chương, <code>next_chapter_number: 2</code> (Đạt)
2	Upload nội dung UTF-16 LE tiếng Việt	TXT có BOM UTF-16 LE	HTTP 200 khi preview	Không lỗi font hoặc mất dấu	Tiêu đề/nội dung tiếng Việt giữ nguyên (Đạt)
3	Không chọn file	Request không có field <code>file</code>	HTTP 400	Không tạo chương, báo thiếu file	HTTP 400, batch insert không được gọi (Đạt)
4	File không có nội dung đọc được	TXT chỉ chứa byte NUL	HTTP 400, <code>success: false</code>	Không import dữ liệu rác	HTTP 400, không tạo chương (Đạt)
5	Sai đuôi file	<code>truyen.exe</code>	HTTP 400	Chỉ chấp nhận <code>.txt</code> , <code>.md</code> , <code>.epub</code>	Middleware từ chối file (Đạt)
6	Đuôi đúng nhưng MIME sai	<code>truyen.txt</code> , MIME <code>image/png</code>	HTTP 400	Kiểm tra cả extension và MIME type	Middleware trả lỗi định dạng (Đạt)
7	File vượt kích thước 25 MB	TXT lớn hơn 25 MB	HTTP 400, <code>LIMIT_FILE_SIZE</code>	Chặn file quá giới hạn	Nhận đúng mã <code>LIMIT_FILE_SIZE</code> (Đạt)
8	Chưa đăng nhập khi import	Không có <code>req.user</code>	HTTP 401	Yêu cầu xác thực	HTTP 401, không truy vấn truyện (Đạt)
9	Uploader không sở hữu truyện	User 99 import vào truyện của user 7	HTTP 403	Chỉ owner/ collaborator/ Admin được thao tác	HTTP 403, không tạo chương (Đạt)
10	Import khi truyện chưa duyệt	Truyện <code>pending</code> , chưa publish	HTTP 409	Chỉ import sau khi Moderator duyệt	HTTP 409, không tạo chương (Đạt)
Còn tiếp ở trang sau					

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
11	Database lỗi khi import	File hợp lệ; DB không khả dụng	HTTP 500, <code>success: false</code>	Không trả số chương thành công giả	HTTP 500, không trả success (Đạt)
12	Tạo truyện mới	Metadata hợp lệ từ Uploader	HTTP 201; truyện chờ duyệt	Lưu uploader và tên tác giả riêng biệt	<code>author_id</code> và <code>author_name</code> lưu đúng (Đạt)
13	Thêm chương trước khi truyện được duyệt	Truyện <code>pending</code>	HTTP 409	Không cho thêm chương	HTTP 409, model tạo chương không được gọi (Đạt)
14	Số chương bị trùng	Chapter number đã tồn tại	HTTP 409, <code>CHAPTER_NUMBER_EXISTS</code>	Không trả lỗi máy chủ chung chung	HTTP 409, đúng mã nghiệp vụ (Đạt)
15	Uploader tự publish truyện	Uploader gọi toggle visibility	HTTP 403	Xuất bản phải qua Moderator/Admin	HTTP 403, không cập nhật truyện (Đạt)
16	Preview file truyện	TXT có heading chương	HTTP 200, danh sách preview	Preview không tạo dữ liệu chương	Có preview, <code>Chapter.createChapter</code> không được gọi (Đạt)
17	Moderator duyệt truyện <code>pending</code>	Story đang trong moderation queue	HTTP 200	Chuyển trạng thái đúng và publish	Truyện được duyệt theo workflow (Đạt)
18	Moderator yêu cầu sửa có lý do	Action request changes và reason hợp lệ	HTTP thành công, có notification	Lưu lý do và thông báo Uploader	Trạng thái và notification được cập nhật (Đạt)
19	Moderator yêu cầu sửa thiếu lý do	Action request changes, reason rỗng	HTTP 400	Bắt buộc có lý do	HTTP 400, không cập nhật truyện (Đạt)
20	Moderator lọc danh sách theo tiêu chí	Status/search/page/limit	Danh sách và global status counts	Lọc đúng; count phản ánh toàn hệ thống	Kết quả lọc và counts đúng (Đạt)
21	Xử lý report avatar	Report avatar gắn với user bị báo cáo	HTTP thành công	Cập nhật đúng boolean/status, không nhầm enum	Tham số cập nhật tách biệt đúng (Đạt)

3.6.5. Nhóm 3: Chức năng người dùng

Các tính năng tương tác của người dùng như tìm kiếm, theo dõi truyện, bình luận, đánh giá, báo cáo vi phạm, tải ảnh đại diện và mở khóa chương bằng Tinh thạch được kiểm chứng thông qua 29 kịch bản (Bảng 3.10). Việc kiểm thử được thực hiện qua các test suite: socialInteractions.test.js, storyDiscoveryRating.test.js, reportController.test.js, chapterUnlock.test.js và uploadCover.test.js.

Bảng 3.10: Test case nhóm chức năng người dùng

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
1	Upload avatar JPG/PNG/WebP hợp lệ	File nhỏ hơn hoặc bằng 5 MB, MIME ảnh	HTTP 200 qua middle-ware	Chấp nhận các ảnh hợp lệ	Cả JPG, PNG, WebP đều được nhận (Đạt)
2	Upload file script giả avatar	../attack.svg, MIME text/html	HTTP 400	Không nhận nội dung không phải ảnh	Middleware từ chối (Đạt)
3	Avatar vượt 5 MB	PNG lớn hơn 5 MB	HTTP 400, LIMIT_FILE_SIZE	Chặn file quá lớn	Nhận đúng mã giới hạn (Đạt)
4	Tìm kiếm kết hợp	Query có dấu/ký tự đặc biệt, category, tag, page	HTTP 200, stories và pagination	Truyền đúng bộ lọc, không crash	Model nhận đúng tham số, HTTP 200 (Đạt)
5	Tìm kiếm không có kết quả	Từ khóa không tồn tại	HTTP 200, stories: []	Danh sách rỗng không phải lỗi	HTTP 200, mảng rỗng (Đạt)
6	Guest xem truyện nhấp	Slug của truyện chưa publish	HTTP 404	Không lộ nội dung chưa duyệt	HTTP 404 (Đạt)
7	Guest kiểm tra trạng thái follow	Không có user/token	HTTP 200, following: false	Endpoint optional auth không trả 401	HTTP 200, false (Đạt)
8	Follow/unfollow lặp lại	Gọi follow hai lần, unfollow hai lần	Các re-request thành công/idempotent	Không gây lỗi hoặc dữ liệu trùng	Controller xử lý các lần gọi ổn định (Đạt)
9	Follow với story ID sai	ID 0, -1, abc	HTTP 400	Chỉ nhận ID nguyên dương	Cả ba trường hợp đều bị từ chối (Đạt)
10	Follow API thất bại, UI tự động rollback	UI cập nhật trước (optimistic); API reject	UI rollback trạng thái	Không để trạng thái giả trên giao diện	Nút follow quay lại trạng thái trước (Đạt)
Còn tiếp ở trang sau					

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
11	Bình luận rỗng hoặc quá dài	Khoảng trắng hoặc hơn 2.000 ký tự	HTTP 400	Không tạo bình luận sai	HTTP 400, model create không được gọi (Đạt)
12	Reply sang thread khác	Parent thuộc story/ chapter khác	HTTP 400	Không gắn reply sai thread	HTTP 400, không tạo comment (Đạt)
13	User xóa bình luận của người khác	User thường không phải owner	HTTP 403	Bảo vệ quyền sở hữu comment	HTTP 403, không xóa (Đạt)
14	Admin xóa bình luận bất kỳ	Admin xóa comment của user khác	HTTP 200	Admin được phép moderation	Comment được xóa (Đạt)
15	Vote sai giá trị	Vote 0, 2, -2	HTTP 400	Chỉ nhận 1 hoặc -1	Cả ba giá trị bị từ chối (Đạt)
16	Rating ngoài khoảng hợp lệ	Rating 0, 6, 1.5, "five"	HTTP 400	Chỉ nhận số nguyên 1-5	Không gọi upsert rating (Đạt)
17	Thêm/cập nhật rating hợp lệ	Rating 5 của user đăng nhập	HTTP 200, summary mới	Upsert rating của đúng user	Model nhận story ID, user ID và rating đúng (Đạt)
18	Xóa rating của chính user	User xóa rating của mình	HTTP 200, summary mới	Không xóa rating của user khác	Model được gọi với đúng user hiện tại (Đạt)
19	Tạo report hợp lệ	User, target story, reason, description	HTTP 201	Tạo report đúng target	Insert đúng dữ liệu (Đạt)
20	Spam report vượt rate limit	Đã gửi đủ giới hạn trong một giờ	HTTP 429	Chặn spam trước khi insert	HTTP 429, không insert report (Đạt)
21	Report avatar từ comment	Comment ID có tác giả	Report gắn reported_user_id	Moderator biết đúng user bị báo cáo	User ID được lấy từ tác giả comment (Đạt)
22	Cập nhật report không tồn tại	Report ID không có trong DB	HTTP 404	Không tạo trạng thái giả	HTTP 404 (Đạt)
23	Hành động report sai loại target	Dùng chapter action cho comment report	HTTP 400	Chặn action không tương thích	HTTP 400, không cập nhật target (Đạt)
24	Mở khóa chương miễn phí	Chapter is_paid: false	HTTP 409, không trừ tiền	Không gọi giao dịch ví	Wallet không được gọi (Đạt)
25	Mở khóa chương trả phí	Chapter khóa; số dư đủ	HTTP 200, CHAPTER_UNLOCKED	Trừ đúng UNLOCK_COST và trả số dư mới	Chương được mở, số dư giảm đúng chi phí cấu hình (Đạt)
Còn tiếp ở trang sau					

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
26	Mở khóa lại hoặc request đồng thời	Wallet báo <code>already_unlocked: true</code>	HTTP 200, cost 0	Không trừ tiền lần hai	CHAPTER_ALREADY_UNLOCKED, cost 0 (Đạt)
27	Số dư không đủ khi mở khóa	Balance nhỏ hơn UNLOCK_COST	HTTP 409, INSUFFICIENT_CRYSTALS	Không mở khóa; trả số dư hiện tại	API trả 409, số dư không thay đổi (Đạt)
28	Database lỗi khi mở khóa	DB error chứa dữ liệu nội bộ	HTTP 500, thông báo chung	Không lộ chi tiết database	UI chỉ hiển thị thông báo chung tiếng Việt (Đạt)

3.6.6. Kiểm thử thủ công bổ sung

Đối với các nghiệp vụ phức tạp liên quan đến trải nghiệm tương tác giao diện (UI/UX), khả năng hiển thị tương thích đa thiết bị và sự tích hợp với các dịch vụ bên ngoài trong điều kiện thực tế, hệ thống được xác minh bổ sung thông qua phương pháp kiểm thử thủ công. Kết quả đánh giá cho 10 kịch bản đặc thù này được tổng hợp chi tiết tại Bảng 3.11, qua đó khẳng định hệ thống đáp ứng đầy đủ các yêu cầu đã đặt ra.

Bảng 3.11: Kịch bản kiểm thử thủ công bổ sung

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
MT01	Xem và đọc truyện public không cần đăng nhập	Guest; có truyện approved; từ khóa tìm kiếm	Trang reader và chi tiết hiển thị bình thường	Không yêu cầu login; URL đúng slug/chương	Hiển thị trang reader đầy đủ, URL chứa slug chuẩn (Đạt)
MT02	Follow truyện và kiểm tra tính nhất quán	User đăng nhập; follow, tải lại, xem danh sách theo dõi	Trạng thái follow đồng bộ DB	Nhất quán với DB; lỗi mạng có rollback	Trạng thái follow/unfollow đồng bộ ngay sau khi tải lại (Đạt)
MT03	Ghi nhận lịch sử đọc đúng user	User đọc nhiều chương; đọc, cuộn, thoát và mở lịch sử	Lịch sử hiển thị đúng	Trở về đúng truyện/chương; không ghi nhầm user	Lịch sử được lưu đúng tài khoản và vị trí chương đã đọc (Đạt)
MT04	Uploader không thể thêm chương/publish khi pending	Uploader; truyện pending, chưa duyệt	HTTP 409/403, thông báo rõ ràng	Bị chặn với thông báo đúng; không đổi DB	Uploader nhận lỗi HTTP 409/403 trên cả UI và API (Đạt)
MT05	Moderator duyệt có/không có lý do	Moderator; truyện pending; lần 1 không nhập lý do, lần 2 nhập lý do	Lần 1: HTTP 400; Lần 2: HTTP 200	Lần đầu bị validate; lần sau cập nhật trạng thái, notify và audit	Bị chặn khi gửi lý do trống; duyệt thành công khi điền đủ lý do (Đạt)
MT06	Rate limit khi gửi report nhiều lần	User; gửi report lặp trong 1 giờ	Report đầu: HTTP 201; lần sau: HTTP 429	Report đầu hợp lệ; request lặp bị hạn chế	API phản hồi HTTP 429 sau khi gửi report dồn dập (Đạt)
MT07	Admin thay đổi role/status người dùng	Admin đăng nhập; chọn user, đổi role/status	UI cập nhật; audit log ghi nhận	UI cập nhật; audit không chứa password/token	Quyền user thay đổi ngay lập tức, audit log ghi nhận đầy đủ (Đạt)
Còn tiếp ở trang sau					

STT	Test Case	Input	Output	Kết quả mong muốn	Kết quả thực tế
MT08	Token hết hạn không gây vòng lặp redirect	Token hết hạn; truy cập trang account/admin	HTTP 401; redirect về /login	API trả 401; frontend xóa phiên, không lặp vô hạn	API trả 401; frontend xóa session và redirect về /login (Đạt)
MT09	Responsive UI trên mobile và desktop	Viewport 375px và desktop; duyệt Home, Reader, Admin	Layout hiển thị đúng theo viewport	Không tràn ngang; nút thao tác được; chữ đủ đọc	Giao diện hiển thị tốt trên các kích thước màn hình mobile/desktop (Đạt)
MT10	Queue job notification với Redis thật	Staging với Redis thật; tạo chương mới đủ điều kiện	Job xử lý; notification được tạo	Job xử lý một lần; notification tạo đúng người nhận	Queue job Notification xử lý thành công qua Redis trên staging (Đạt)

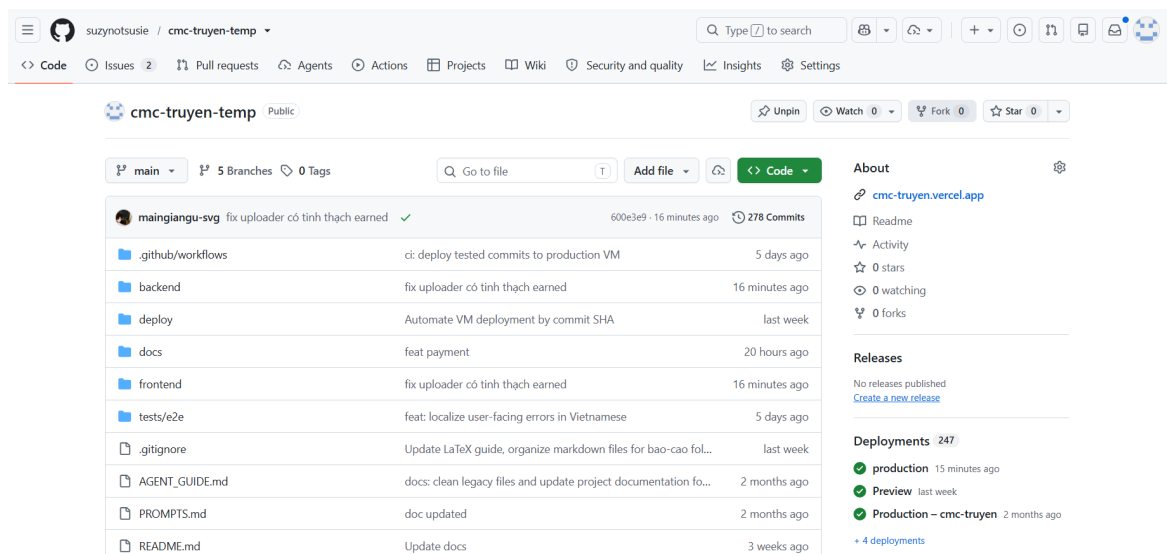
3.7. Kết luận chương

Chương 3 đã trình bày kiến trúc SPA + REST, thiết kế giao diện theo vai trò, cách tổ chức frontend, backend, cơ sở dữ liệu, triển khai và kiểm thử. Phần đánh giá, hạn chế và hướng hoàn thiện được tổng hợp trong kết luận cuối cùng của báo cáo.

Chương 4. CỘNG TÁC TRÊN GITHUB VÀ SỬ DỤNG CÔNG CỤ AI

4.1. Kho mã nguồn và quy ước cộng tác

Kho mã nguồn công khai được kiểm tra tại <https://github.com/suzynotsusie/cmc-truyen-temp>. Hình 4.1 là ảnh chụp trực tiếp trang kho mã nguồn dùng làm minh chứng.



Hình 4.1: Kho mã nguồn GitHub của dự án CMC Truyen

Nhóm làm việc theo vòng lặp: chia nhỏ yêu cầu; tạo nhánh hoặc task; commit theo thay đổi có ý nghĩa; mở pull request khi cần review/tích hợp; chạy test tự động; chỉ đưa commit đạt yêu cầu vào nhánh chính. README là điểm vào cho người mới, trong khi docs/product, docs/technical, docs/testing và docs/ai-logs lưu bằng chứng chi tiết.

4.2. Issues, commits, pull requests, milestones và releases

Bảng 4.1: Đối chiếu các cơ chế cộng tác trên GitHub

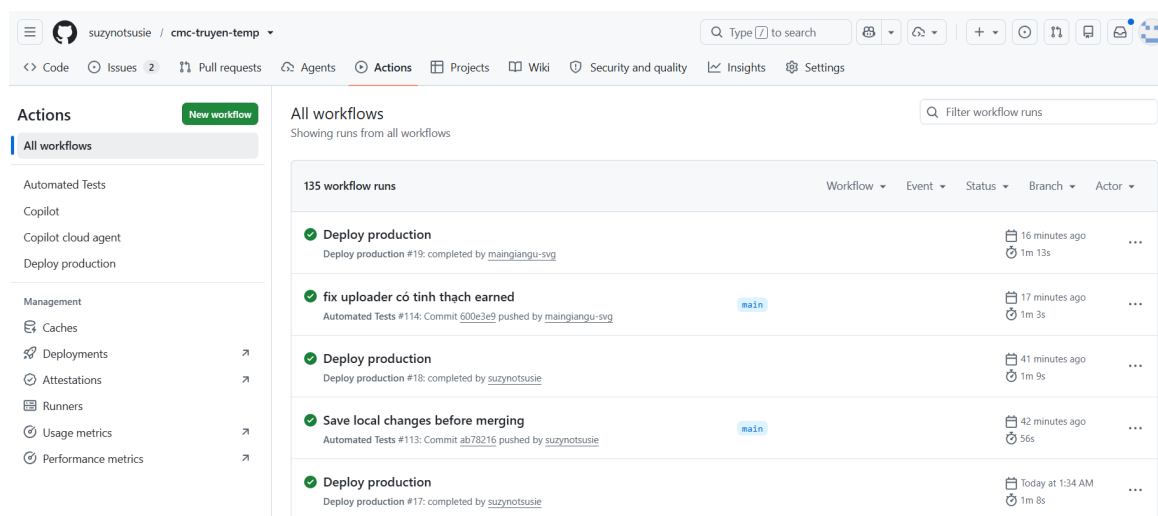
Cơ chế	Cách nhóm đã sử dụng / bằng chứng	Đánh giá và hành động đã thực hiện
README và tài liệu	README mô tả tính năng, kiến trúc, môi trường, lệnh chạy/test, route và nguyên tắc đóng góp. Product vision, persona, story và backlog nằm trong docs/product/REQUIREMENTS.md.	Tài liệu đầy đủ, minh bạch và làm điểm mốc thống nhất cho cả dự án.
Commits	Lịch sử có hơn 276 commit trên trang GitHub. Ví dụ: 395f087 (release v1.0.0), 80fdb59 (release v1.1.0), 977ccb3 (release v1.2.0), ab78216 (release v2.0.0).	Có khả năng truy vết thay đổi minh bạch theo từng mốc phát triển.
Pull requests	13 PR đã đóng, 0 PR mở. PR #12 sửa auth/API production; PR #13 tự động hóa deploy; các PR #7, #9, #10 liên quan đến test và code review.	PR cung cấp điểm review và tích hợp chặt chẽ. Đóng PR không đạt đảm bảo chất lượng nhánh chính.
Issues	Hệ thống đã quản lý đầy đủ 18 GitHub Issues từ #26 đến #43, gắn nhãn (labels), phân công công việc (assignees) và liên kết chặt chẽ với các Milestones và PR.	Đạt chuẩn tiêu chí quản lý task minh bạch trên GitHub.
Milestones	Nhóm xây dựng 4 Milestones chính: (1) Core Reading Platform (Issues #26–#30, #32); (2) Engagement & Search (Issues #31, #33–#36); (3) Moderation & Safety (Issues #37–#40); (4) AI & Analytics (Issues #41–#43).	Quản lý phạm vi phát triển dự án theo đúng quy trình agile/scrum.

Cơ chế	Cách nhóm đã sử dụng / bằng chứng	Đánh giá và hành động đã thực hiện
Releases	Đã đóng gói 4 phiên bản chính thức trên GitHub: v1.0.0 (commit 395f087), v1.1.0 (commit 80fdb59), v1.2.0 (commit 977ccb3) và v2.0.0 (commit ab78216) với Release Notes chi tiết.	Truy vết phiên bản hệ thống hoàn chỉnh và chuyên nghiệp.

Quy trình quản lý trên GitHub được nhóm thực hiện đồng bộ từ việc tạo Issues, gom nhóm vào các Milestones tương ứng, phát triển theo nhánh và kiểm thử tự động, tới việc tạo Tags và Releases đánh dấu mốc hoàn thiện sản phẩm. Dữ liệu đối chiếu hoàn toàn trùng khớp với lịch sử kho mã nguồn.

4.3. Code review, kiểm thử tự động và DevOps

Workflow `.github/workflows/tests.yml` chạy khi push, pull request hoặc kích hoạt thủ công, gồm backend Jest, frontend Vitest/build và Playwright E2E. Workflow `deploy-production.yml` chỉ deploy khi bộ test thành công trên nhánh mặc định và truyền đúng commit SHA đến máy chủ. Tại thời điểm kiểm tra, GitHub Actions hiển thị 131 workflow run; Automated Tests #112 và Deploy production #17 đều hoàn tất thành công cho commit mới nhất lúc đó.



Hình 4.2: GitHub Actions với các lần chạy test và deploy thành công

Thiết kế trên tạo chuỗi kiểm soát “commit – test – deploy đúng SHA”, giảm nguy cơ triển khai một phiên bản khác với phiên bản đã kiểm thử. Bí mật SSH được lấy từ GitHub Secrets, known host được ghim và job deploy chỉ có quyền đọc nội dung kho. Tuy vậy, dự án vẫn cần môi trường staging có PostgreSQL/Redis/dịch vụ ngoài thật, kiểm thử tải và security audit trước khi xem quy trình là production-ready [13, 10].

4.4. Nhật ký sử dụng công cụ và tác tử AI

Nhóm lưu nhật ký theo tuần trong docs/ai-logs, hướng dẫn prompt trong PROMPTS.md, quy tắc review tại AGENT_GUIDE.md và docs/technical/AI_USAGE_POLICY.md. Nguyên tắc xuyên suốt là AI đóng vai trò hỗ trợ; thành viên phải đọc, kiểm tra và chịu trách nhiệm đối với output trước khi merge.

Bảng 4.2: AI gợi ý, phân nhóm kiểm chứng/sửa/loại bỏ và quyết định của nhóm

Hoạt động AI	Gợi ý ban đầu	Nhóm kiểm chứng, sửa hoặc loại bỏ	Quyết định/bằng chứng
Phân tích MVP bằng RICE	Ưu tiên trình đọc sạch, tùy chỉnh UI và bookmark; đề xuất crawler và gamification ở mức thấp.	Nhóm đối chiếu thời gian/nguồn lực, giữ ba năng lực cốt lõi; loại crawler và gamification khỏi MVP.	Nhật ký week-02.md, PRODUCT_ANALYSIS.md.
Đề xuất cấu trúc React	Chuyển mã giao diện cũ thành component, service và route React/Vite; dùng mock để dựng prototype.	Nhóm giữ redirect tương thích, kiểm tra route và tích hợp API thật theo từng bước, không giữ mock như bằng chứng production.	Nhật ký week-03.md, CODEBASE_OVERVIEW.md.

Hoạt động AI	Gợi ý ban đầu	Nhóm kiểm chứng, sửa hoặc loại bỏ	Quyết định/bằng chứng
Soạn user story và acceptance criteria	AI chuẩn hóa câu “Là một... tôi muốn... để...” và checklist cho AI summary, recommendation, moderation, report.	Nhóm sửa ràng buộc theo Express/PostgreSQL hiện có và kiểm tra bằng test/source.	Nhật ký week-04.md, PROMPTS.md.
Copilot sửa test/code	Bot tạo các PR về StoryCard test và moderator test.	PR #7, #9, #10 được review và merge; PR nháp #5, #6 không đạt bị đóng.	Lịch sử Pull Request và GitHub Actions.
Tích hợp AI sản phẩm	Gợi ý dùng Groq/Gemini cho tóm tắt và đề xuất truyện.	Nhóm thêm cache, timeout, fallback, validation ID, không đưa API key xuống frontend và không để AI tự quyết định khóa tài khoản/duyet truyện.	Service aiService.js, tài liệu AI_PERSONALIZATION.md, AI_USAGE_POLICY.md.

Những quyết định nhóm tự chịu trách nhiệm gồm lựa chọn phạm vi MVP, kiến trúc frontend/backend tách rời, PostgreSQL/Redis, phân quyền, tiêu chuẩn nghiệm thu, việc merge/đóng PR và chấp nhận các hạn chế còn lại. AI không thay thế quyết định sản phẩm, review bảo mật hoặc phán quyết kiểm duyệt có tác động đến người dùng.

Chương 5. LIÊN HỆ LÝ THUYẾT ENGINEERING SOFTWARE PRODUCTS

5.1. Phương pháp đối chiếu

Chương này đối chiếu quyết định của nhóm với các khái niệm trong *Engineering Software Products: An Introduction to Modern Software Engineering* của Ian Sommerville [11]. Mỗi mục đi theo mạch: lý thuyết và chương tham chiếu; cách áp dụng có bằng chứng; phản biện về mức phù hợp hoặc lý do không áp dụng đầy đủ. Cách trình bày này tránh việc chỉ liệt kê thuật ngữ mà không gắn với quyết định thực tế.

Bảng 5.1: Đối chiếu quyết định của nhóm với lý thuyết Sommerville

#	Khái niệm / Chương (Sommerville)	Quyết định của nhóm	Link đến minh chứng (trên GitHub)	Phản biện (phù hợp / không áp dụng & vì sao)
1	Product vision & MVP (Ch.2)	Viết product vision 1 câu; chọn các chức năng lõi cho bản MVP (xác thực, core reader, tùy chỉnh UI, CRUD truyện/chương)	Issue #26 (vision), Milestone “Core Reading Platform”, Release v1.0.0	Phù hợp: nhóm 4 người, thời gian ngắn nên cần giới hạn phạm vi để hoàn thiện trình đọc cốt lõi sớm nhất.
2	Persona & user story (Ch.3)	Dựng 2 persona; xây dựng 18 user story kèm acceptance criteria chuẩn hóa	Issues #26–#43, file REQUIREMENTS.md	Phù hợp: giúp bám sát nhu cầu thực tế của người đọc, uploader và moderator thay vì đặc tả cứng.
3	Kiến trúc: monolith vs microservices (Ch.4–5)	Chọn kiến trúc Modular Monolith (Frontend React SPA tách khỏi REST API Express, DB PostgreSQL & Redis Queue)	PR #12 (API architecture), file README.md	Không dùng microservices: quy mô dự án và nhóm nhỏ, chia dịch vụ sớm sẽ gây phức tạp thừa về hạ tầng và chi phí vận hành.
4	REST API (Ch.6)	Thiết kế API theo chuẩn RESTful cho toàn bộ các module (truyện, chương, tương tác, moderation, AI)	Issues #26–#30, #43, PR #12	Phù hợp: client web SPA và worker/AI dùng chung một API thống nhất, dễ nâng cấp và kiểm thử.
5	Kiểm thử & lập trình tin cậy (Ch.7–8)	Viết unit test cho phần lõi, E2E test cho luồng chính và review mã nguồn qua Pull Request	Issues #37–#40, Release v1.2.0, PR #7, PR #9, PR #10	Phù hợp: bảo vệ phần logic lõi và phòng ngừa lỗi tái phát; bộ test tự động CI đạt 100% pass.
6	DevOps / CI-CD (Ch.9–10)	Chạy test tự động khi push/PR (tests.yml), tự động deploy production và đóng gói các Release phiên bản	Workflows tests.yml, deploy-production.yml, Releases v1.0.0 , v1.1.0 , v1.2.0 , v2.0.0	CI/CD kèm Release tagging giúp kiểm soát chất lượng và truy vết phiên bản minh bạch trước khi xuất bản.

5.2. Phân tích sâu các quyết định chính

5.2.1. Giới hạn MVP và quản lý phạm vi

Theo tư duy sản phẩm, MVP không phải là phiên bản chứa mọi chức năng ở chất lượng thấp mà là tập năng lực nhỏ nhất đủ kiểm chứng giá trị. Ba năng lực đọc sạch, tùy chỉnh giao diện và khôi phục tiến độ tạo thành một vòng giá trị hoàn chỉnh: người dùng tìm được nội dung, đọc thuận tiện và có lý do quay lại. Việc đặt AI ở lớp mở rộng là hợp lý vì hệ thống vẫn có giá trị khi nhà cung cấp AI lỗi hoặc chưa cấu hình key.

Phản biện quan trọng là trạng thái “Completed” trong backlog chỉ phản ánh mức hoàn thành kỹ thuật được nhóm ghi nhận, không tự động chứng minh mức chấp nhận của người dùng. Tiêu chí “dưới 30 giây” cần được đo qua usability test có kịch bản, thiết bị và số người tham gia cụ thể; nếu chưa có số liệu thì chỉ nên coi là mục tiêu.

5.2.2. Kiến trúc đủ dùng thay vì microservices sớm

Frontend/backend tách rời cho phép React và API triển khai độc lập, trong khi backend vẫn là một hệ thống có module thay vì nhiều dịch vụ mạng. Cách này giảm chi phí discovery, observability, giao dịch phân tán và đồng bộ schema. PostgreSQL phù hợp với quan hệ giữa user, story, chapter, interaction và moderation; Redis/BullMQ chỉ được dùng nơi cần queue/tác vụ nền.

Nhóm không áp dụng microservices vì chưa có bằng chứng về tải hoặc ranh giới tổ chức đòi hỏi tách dịch vụ. Quyết định này phù hợp với nguyên tắc tránh độ phức tạp không tạo giá trị. Nếu số lượng worker, lưu lượng AI hoặc moderation tăng, nhóm có thể tách theo dữ liệu đo được thay vì theo xu hướng công nghệ.

5.2.3. AI-assisted development và trách nhiệm kỹ thuật

Công cụ AI hỗ trợ tăng tốc quá trình phân tích yêu cầu, xây dựng mã mẫu, tự động hóa tiêu chí nghiệm thu và khởi tạo các kịch bản kiểm thử. Lịch sử PR thể hiện các đề xuất mã nguồn từ Copilot luôn trải qua quy trình đánh giá (Code Review) và chạy bộ test tự động trước khi được tích hợp vào nhánh chính; các đề xuất không đạt tiêu chuẩn đều bị loại bỏ. Quy trình này đảm bảo đội ngũ phát triển kiểm soát hoàn toàn chất lượng phần mềm.

Các quyết định thiết kế cốt lõi—bao gồm giới hạn phạm vi MVP, kiến trúc phân tầng, phân quyền truy cập, nguyên tắc bảo mật và điều kiện nghiệm thu—đều do nhóm phát triển trực tiếp quyết định. Đơn cử đối

với các tính năng tích hợp AI trong sản phẩm, hệ thống áp dụng cơ chế cache, timeout, fallback và không cho phép AI tự động đưa ra các quyết định quản trị có tác động lớn. Cách tiếp cận này đáp ứng tốt các nguyên tắc về lập trình tin cậy và khả năng kiểm soát hệ thống [12, 13].

Chương 6. KẾT QUẢ THỰC HIỆN

6.1. Sản phẩm và kịch bản MVP

Sản phẩm hiện có tại <https://cmc-truyen.vercel.app>; mã nguồn và lịch sử cộng tác được lưu tại <https://github.com/suzynotsusie/cmc-truyen-temp>.

Kịch bản MVP xuyên suốt là: người dùng mở trang chủ, tìm hoặc chọn truyện, xem chi tiết, mở chương, tùy chỉnh chế độ đọc, chuyển chương; sau khi đăng nhập, tiến độ được lưu để lần sau tiếp tục. Kịch bản mở rộng đi qua toàn bộ chuỗi sản phẩm: Uploader tạo truyện, Moderator duyệt, User đọc/tương tác, Admin xem báo cáo và audit.

Bảng 6.1: Đối chiếu kết quả MVP với mục tiêu sản phẩm

Mục tiêu	Kết quả và bằng chứng	Đánh giá
Trình đọc sạch	Route đọc chương, điều hướng trước/sau, preview chương khóa và giao diện không chèn quảng cáo trong nội dung đọc; ảnh màn hình đọc ở Chương 3.	Đạt về chức năng
Tùy chỉnh trải nghiệm	Hỗ trợ giao diện sáng/tối và các thiết lập đọc; trạng thái UI được quản lý ở frontend.	Đạt về chức năng
Lưu và khôi phục tiến độ	API/lịch sử đọc và trang tài khoản hỗ trợ ghi nhận chương đã đọc, tiếp tục theo tài khoản.	Đạt về chức năng
Luồng nội dung	Uploader tạo/nhập truyện, quản lý chương; Moderator duyệt/yêu cầu sửa/từ chối; có trạng thái và audit.	Đạt phần chính
Quản trị và an toàn	JWT, RBAC, validation, rate limit, moderation, report, audit log và quản lý user.	Đạt phần chính
AI hỗ trợ	Tóm tắt chương và gợi ý cá nhân hóa có cache, fallback và giới hạn tin cậy.	Hoàn thành theo backlog

6.2. Đánh giá chức năng, cơ sở dữ liệu và kiến trúc

Về chức năng, hệ thống bao phủ năm vai trò Guest, User, Uploader, Moderator và Admin. Ma trận quyền, các nhánh lỗi và ánh xạ endpoint/source đã được trình bày ở Chương 2; giao diện, triển khai frontend/backend, database và kiểm thử được trình bày ở Chương 3.

Về dữ liệu, PostgreSQL lưu user, story, chapter, tag, lịch sử đọc, follow, rating, comment, report, audit và dữ liệu liên quan; transaction được dùng cho nghiệp vụ cần tính nhất quán như mở khóa chương. Redis/BullMQ hỗ trợ queue/worker và PostgreSQL vẫn là nguồn dữ liệu bền vững. ERD và schema chính đã được chèn trong Chương 3.

Về kiến trúc, hệ thống đạt mục tiêu tách SPA khỏi REST API và tách tác vụ nền khỏi request chính. Điểm mạnh là ranh giới trách nhiệm tương đối rõ, API có middleware xác thực/phân quyền và pipeline CI kiểm tra cả backend, frontend, E2E trước deploy. Kiến trúc này phù hợp quy mô hiện tại hơn microservices, nhưng vẫn cần quan sát queue, lỗi dịch vụ ngoài và độ trễ AI khi vận hành thật.

6.3. Hạn chế được xác nhận

- Product backlog còn PB17–PB19 ở trạng thái đang hoàn thiện: báo cáo/analytics, auto moderation và xử lý vi phạm cần thêm kiểm chứng trên hạ tầng thật.
- E2E hiện mock nhiều dependency; chưa thay thế staging test với PostgreSQL, Redis, email, storage và OAuth thật.
- Cần tiếp tục tối ưu bundle frontend, teardown queue, kiểm thử tải, security audit và accessibility audit.

6.4. Hướng phát triển

Ưu tiên gần nhất là hoàn tất PB17–PB19 và nghiệm thu trên staging; bổ sung quan sát Redis/BullMQ, log có cấu trúc và cảnh báo; tối ưu bundle bằng lazy loading; tăng kiểm thử tích hợp, tải, bảo mật và accessibility.

Về sản phẩm, nhóm chỉ mở rộng sau khi có dữ liệu: nâng chất lượng gợi ý từ lịch sử thật, thêm công cụ Moderator theo rủi ro, và cải thiện trải nghiệm đọc trên thiết bị di động. Các quyết định AI có ảnh hưởng đến nội dung hoặc tài khoản vẫn phải có human review, khả năng giải thích và cơ chế khiếu nại.

TÀI LIỆU THAM KHẢO

- [1] Nhóm phát triển CMC Truyện, *Đặc tả yêu cầu và phạm vi chức năng dự án CMC Truyện*, tài liệu nội bộ trong thư mục docs, phiên bản dùng để nộp, 2026.
- [2] Nhóm phát triển CMC Truyện, *Tài liệu kiến trúc, API và cấu hình triển khai dự án CMC Truyện*, tài liệu nội bộ trong kho mã nguồn, phiên bản dùng để nộp, 2026.
- [3] Meta Open Source, *React Documentation – Learn React*, <https://react.dev/learn>
- [4] VoidZero Inc. và cộng đồng Vite, *Vite Guide – Getting Started*, <https://vite.dev/guide/>
- [5] OpenJS Foundation, *Express.js Guide – Routing*, <https://expressjs.com/en/guide/routing/>
- [6] PostgreSQL Global Development Group, *PostgreSQL Documentation – Constraints*, <https://www.postgresql.org/docs/current/ddl-constraints.html>
- [7] M. Jones, J. Bradley và N. Sakimura, *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force, 2015, <https://datatracker.ietf.org/doc/html/rfc7519>.
- [8] Redis, *Redis Documentation*, <https://redis.io/docs/latest/>
- [9] Taskforce.sh, *BullMQ Documentation*, <https://docs.bullmq.io/>
- [10] OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, <https://owasp.org/www-project-top-ten/>
- [11] Ian Sommerville, *Engineering Software Products: An Introduction to Modern Software Engineering*, Pearson Education, 2020.
- [12] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.

- [13] Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill, 2020.